

coupledFoam: a block-coupled pressure–velocity solver with single-precision block-multigrid linear algebra for steady incompressible RANS in OpenFOAM

Jonas Pangerl

with implementation assistance by Claude (Anthropic)

Generated at commit `dca2666`

Abstract

We describe **coupledFoam**, a steady, incompressible, Reynolds-averaged Navier–Stokes solver for OpenFOAM v2606 in which velocity and pressure are solved simultaneously as one 4×4 -block system per cell, while the turbulence model is solved segregated and unmodified. The block matrix and all Krylov and multigrid vectors are stored in single precision; the right-hand side of every outer iteration is the residual $b - Ax$ of the double-precision discretisation, and every reduction is accumulated in double. The outer iteration is therefore a mixed-precision defect correction whose attainable accuracy is set by the double-precision residual, not by the single-precision solve. The nonlinear iteration is globalised by pseudo-transient continuation with local, solution-limited time steps and an adaptive CFL number, a physicality line search, two-tier cell sets that locally reduce the scheme order and the time step, a sentinel with rollback, and an exact restart. The linear systems are solved by flexible GMRES preconditioned with an additive-correction block algebraic multigrid that reuses OpenFOAM’s agglomeration and supports V-, F-, W- and Notay–Vassilevski K-cycles, with an Eisenstat–Walker inner tolerance. On the lid-driven cavity at $Re = 100$ the solver reproduces the centreline profiles of the native segregated solver **simpleFoam** to a relative L_2 difference of 2.9×10^{-6} in 57 outer iterations. Performance against tuned native SIMPLE and SIMPLEC is measured in wall-clock time and CPU-hours to an identical convergence criterion, because the target is the fastest solver, not the one with the fewest iterations.

Contents

1	Introduction	2
2	Discretisation and the coupled system	4
2.1	Governing equations and notation	4
2.2	Momentum rows	4
2.3	Continuity row and Rhie–Chow interpolation	5
2.4	Block structure	5
2.5	Boundary conditions	6
2.6	Implicit MRF	6
2.7	Non-orthogonal correction with a per-face limiter	7
2.8	Turbulence coupling	7
3	Solution algorithm	7
3.1	Increment formulation and the double-precision residual	7
3.2	Residual norms	7
3.3	Outer iteration	8
3.4	Pseudo-transient continuation and local time step	8

3.5	Solution-limited local time step	9
3.6	Physicality line search	9
3.7	Start-up	9
3.8	Remediation cell sets	9
3.9	Sentinel, rollback and failure handling	10
3.10	Restart	10
3.11	Eisenstat–Walker forcing	11
3.12	Anderson acceleration	11
4	Linear algebra	11
4.1	Storage and precision	11
4.2	Krylov solvers	12
4.3	Block additive-correction AMG	13
5	Implementation in OpenFOAM	15
6	Verification and validation	16
6.1	Test cases	16
6.2	Results	17
6.3	Flow fields	20
7	Performance	41
7.1	Benchmark protocol	41
7.2	Results	42
7.3	Speed-up measured on the test runs	46
8	Discussion and limitations	46
8.1	What single precision requires	46
8.2	Deferred correction at high CFL	46
8.3	Other limitations	50
8.4	Phase 2	50
9	Conclusions	50

1 Introduction

Most finite-volume solvers for incompressible flow in production use, including the standard OpenFOAM solvers, are *segregated*: the momentum equations are solved with a lagged pressure, and a pressure (correction) equation derived from continuity restores mass conservation. The SIMPLE algorithm of Patankar and Spalding [27] and its consistent variant SIMPLEC of Van Doormaal and Raithby [35] are the prototypes. Their cost per iteration is low and their memory footprint is small, but the pressure–velocity coupling is only approximated by the splitting, so the number of outer iterations grows with mesh size and with the strength of the coupling, and under-relaxation factors have to be tuned for every class of flow.

Coupled solvers discretise momentum and continuity into one linear system and solve for velocity and pressure simultaneously. The block-implicit multigrid method of Vanka [36] is an early example on structured grids. Darwish, Sraj and Moukalled [8, 9] formulated a coupled pressure-based finite-volume solver with Rhie–Chow interpolation [30] on structured and unstructured grids. Mangani, Buchmayr and Darwish brought this approach to OpenFOAM for steady incompressible turbulent flow [18], extended it to rotating reference frames [19] and to transient flows in an ALE formulation with implicit boundary treatment [20]; their block system is solved with an additive-correction algebraic multigrid in the spirit of Hutchinson and

Raithby [14] and Weiss et al. [38]. Uroić and Jasak developed a block-selective algebraic multi-grid (SAMG) for the implicitly coupled pressure–velocity system in foam-extend [33] and its parallelisation [34]. ICSFoam [26] is an OpenFOAM library for implicitly coupled, primarily density-based simulations of high-speed flows. The price of the coupled formulation is memory: a 4×4 block per cell and per face instead of scalar matrices, and a linear system that is indefinite and non-symmetric.

A coupled solver for steady problems is naturally combined with pseudo-transient continuation (PTC), in which a pseudo-time derivative regularises the Jacobian and is removed as the solution converges. Kelley and Keyes [15] analysed the convergence of PTC; Ceze and Fidkowski [5–7] developed robust CFL strategies (SER, EXP, RDM and a modified RDM) and a physicality-based line search for implicit high-order RANS solvers. Mulder and Van Leer’s switched evolution relaxation [23] is the classical CFL strategy. Inexact solves are controlled by forcing terms in the sense of Eisenstat and Walker [11, 17].

Single precision halves the memory traffic of sparse linear algebra, which is the bottleneck of memory-bound solvers, but it cannot on its own deliver the residual levels required for converged steady solutions. Mixed-precision iterative refinement resolves this: the correction equation is solved in low precision while the residual is evaluated in high precision, and the limiting accuracy is that of the residual [3, 4]. The increment formulation of a nonlinear outer iteration has exactly this structure.

Contributions. This paper documents coupledFoam as implemented; every deviation from its specification is recorded in the project file `DECISIONS.md` (items D-001 to D-032, referenced below). The contributions are:

1. A clean-room block-coupled pressure–velocity solver for OpenFOAM v2606 (Section 2) in which all boundary conditions are made implicit through OpenFOAM’s own patch linearisation, the continuity row and the post-solve flux are evaluated by the same face formula, and implicit MRF is extracted from the native MRF operator.
2. A precision design (Sections 3.1 and 4) with float block storage and double residuals and reductions on a double-precision OpenFOAM build, together with the observations that made it work in practice: a strong pressure reference for closed domains, a best-iterate Krylov return and the treatment of failed linear solves.
3. A globalisation layer (Section 3): PTC with a local solution-limited time step, mRDM/EXP/SER CFL control, physicality line search, static, dynamic and zonal remediation sets, sentinel and rollback, exact restart, Eisenstat–Walker forcing and optional Anderson acceleration.
4. A block additive-correction AMG (Section 4.3) on native OpenFOAM agglomeration with V-, F-, W- and K-cycles, a coarsening-ratio rule derived from the cycle cost, a dense coarsest-level LU, processor agglomeration and a cycle-efficiency controller.
5. A verification and benchmark protocol (Sections 6 and 7) that compares against tuned native solvers in wall-clock time and CPU-hours to an identical convergence criterion.

2 Discretisation and the coupled system

2.1 Governing equations and notation

We solve the steady incompressible RANS equations for the velocity $\mathbf{U}$ and the kinematic pressure p ,

$$\nabla \cdot (\mathbf{U}_r \mathbf{U}) - \nabla \cdot (\nu_{\text{eff}} \nabla \mathbf{U}) - \nabla \cdot (\nu_{\text{eff}} \text{dev2}(\nabla \mathbf{U}^T)) + \nabla p + \boldsymbol{\Omega} \times \mathbf{U} = 0, \quad (1)$$

$$\nabla \cdot \mathbf{U} = 0, \quad (2)$$

with $\nu_{\text{eff}} = \nu + \nu_t$, $\text{dev2}(\mathbf{T}) = \mathbf{T} - \frac{2}{3} \text{tr}(\mathbf{T}) \mathbf{I}$ as in OpenFOAM's linear viscous stress, and, inside multiple-reference-frame (MRF) zones rotating with $\boldsymbol{\Omega}$, the absolute velocity $\mathbf{U}$ convected by the relative velocity $\mathbf{U}_r$ (Section 2.6); outside MRF zones $\mathbf{U}_r = \mathbf{U}$ and $\boldsymbol{\Omega} = 0$. The turbulence quantities (k , ω , ν_t) are frozen during the coupled solve and updated afterwards (Section 2.8).

For a cell P with volume V_P and face f shared with neighbour N , we use the face area vector $\mathbf{S}_f$ (pointing out of the owner), $|\mathbf{S}_f|$, the interpolation weight w_f of the owner, the face flux ϕ_f ($\text{m}^3 \text{s}^{-1}$), the non-orthogonal delta coefficient $\Delta_f = 1/(\mathbf{n}_f \cdot \mathbf{d}_f)$ of OpenFOAM, and the orientation $s_{Pf} = +1$ if P owns f and -1 otherwise. The weight of cell P on face f is $w_{Pf} = w_f$ for the owner and $1 - w_f$ for the neighbour. The unknowns per cell are $\mathbf{x}_P = (u, v, w, p)^T$.

2.2 Momentum rows

Convection with deferred correction. The implicit convection operator is OpenFOAM's `fvm::div` with a first-order upwind scheme (key `div(phi,U)_upwind`) on the flux of the previous iteration: owner diagonal $+\max(\phi_f, 0)$, upper coefficient $-\max(-\phi_f, 0)$, lower coefficient $-\max(\phi_f, 0)$, neighbour diagonal $+\max(-\phi_f, 0)$. The user's high-order scheme (key `div(phi,U)`, default `linearUpwind grad(U)`) enters by deferred correction [12, 16]:

$$\mathbf{b}_P^{\text{DC}} = -\beta_P V_P \left[(\nabla \cdot (\phi \mathbf{U}))_P^{\text{HO}} - (\nabla \cdot (\phi \mathbf{U}))_P^{\text{UD}} \right], \quad \beta_P \in [0, 1], \quad (3)$$

where both divergences are explicit evaluations (`fvc::div`) with the respective scheme. The per-cell blending factor β_P is 0 during start-up and in remediated cells and 1 otherwise (Sections 3.7 and 3.8); if $\beta \equiv 0$ the high-order evaluation is skipped.

Diffusion. The implicit diffusion operator is `fvm::laplacian` with the *uncorrected* scheme, i.e. the orthogonal part $\nu_f |\mathbf{S}_f| \Delta_f (\mathbf{U}_N - \mathbf{U}_P)$ with ν_f the linearly interpolated ν_{eff} . The non-orthogonal part is added explicitly with a per-face limiter (Section 2.7), and the transpose stress $\nabla \cdot (\nu_{\text{eff}} \text{dev2}(\nabla \mathbf{U}^T))$ is explicit, exactly as in the native linear viscous stress. The solver assembles the diffusion itself instead of calling the turbulence model's `divDevSigma`, because the latter takes its non-orthogonal limiter from `fvSchemes` and cannot vary it per cell (D-014).

Pressure gradient. The pressure gradient is the Green–Gauss sum $\sum_f s_{Pf} \mathbf{S}_f p_f$ with $p_f = w_f p_P + (1 - w_f) p_N$, so face f contributes $s_{Pf} w_{Pf} \mathbf{S}_f$ to column 3 of the diagonal block of P and $s_{Pf} w_{Nf} \mathbf{S}_f$ to column 3 of the off-diagonal block (P, N) .

Pseudo-time term. Each momentum diagonal receives $V_P / \Delta t_P$ (Section 3.4); in increment form (Section 3.1) the old-time value is the current iterate, so the term contributes only to the matrix, never to the residual, and cannot alter the steady solution.

2.3 Continuity row and Rhie–Chow interpolation

Let a_P^c , $c \in \{u, v, w\}$, be the three momentum diagonal coefficients after all momentum contributions *including* the pseudo-time term, and

$$\bar{a}_P = \frac{1}{3}(a_P^u + a_P^v + a_P^w), \quad D_P = \frac{V_P}{\max(\bar{a}_P, \varepsilon_v)}, \quad D_f = w_f D_P + (1 - w_f) D_N. \quad (4)$$

The face flux is the Rhie–Chow flux [30]

$$\phi_f = \mathbf{S}_f \cdot (w_f \mathbf{U}_P + (1 - w_f) \mathbf{U}_N) - g_f (p_N - p_P) + q_f, \quad g_f = D_f |\mathbf{S}_f| \Delta_f, \quad (5)$$

$$q_f = D_f \left[(w_f \nabla p_P + (1 - w_f) \nabla p_N) \cdot \mathbf{S}_f - |\mathbf{S}_f| c_f(p) \right], \quad (6)$$

where $c_f(p)$ is the limited non-orthogonal correction of the face-normal pressure gradient (Section 2.7). The implicit face-gradient term $-g_f(p_N - p_P)$ forms a Laplacian block in the pressure column of the continuity row; the “averaged cell gradient” term q_f is explicit. The continuity row of cell P is the discrete $\sum_f s_{Pf} \phi_f = 0$ with ϕ_f from (5), i.e. divergence entries $s_{Pf} w_{Pf} \mathbf{S}_f^T$ (diagonal block) and $s_{Pf} w_{Nf} \mathbf{S}_f^T$ (off-diagonal block) in columns 0–2, the Laplacian $+g_f$ on the diagonal and $-g_f$ off the diagonal in column 3, and $-s_{Pf} q_f$ on the right-hand side. No pseudo-time term is added to the continuity row.

The matrix row and the post-solve flux update are built from the same face formula (5) with the same D_f . Consequently, after the update $\sum_f s_{Pf} \phi_f$ equals, cell by cell, the continuity residual of the assembled row evaluated at the new state (with the explicit term q_f refreshed): the mass imbalance of the output flux is exactly the continuity part of the outer residual and vanishes with it. The dimensionally consistent coefficient $D_f |\mathbf{S}_f| \Delta_f$ (rather than $D_f |\mathbf{S}_f|^2 \Delta_f$) and the sign convention of an M-matrix Laplacian are recorded in D-003.

Boundary faces of the Rhie–Chow terms. On physical boundary faces $D_f = D_P$ where the pressure boundary condition fixes the value (`fixesValue()`, e.g. `fixedValue`, `totalPressure`), and $D_f = 0$ elsewhere, since an averaged-gradient term on a zero-gradient wall would create a spurious wall flux (D-013). On symmetry, symmetry-plane and wedge patches the face flux is identically zero and the patch contributes nothing to the continuity row. Processor faces use the interpolated D_f and are implicit; cyclic and cyclicAMI faces use the interpolated D_f with lagged neighbour values.

2.4 Block structure

With the notation above, the diagonal block of cell P and the off-diagonal block of an internal face f between P and N are

$$\mathbf{A}_{PP} = \begin{pmatrix} a_P^u & 0 & 0 & \gamma_P^x \\ 0 & a_P^v & 0 & \gamma_P^y \\ 0 & 0 & a_P^w & \gamma_P^z \\ \delta_P^x & \delta_P^y & \delta_P^z & \sum_f g_f \end{pmatrix} + V_P \begin{pmatrix} [\boldsymbol{\Omega}]_\times & 0 \\ 0 & 0 \end{pmatrix}, \quad \mathbf{A}_{PN} = \begin{pmatrix} a_{PN} l_3 & s_{Pf} w_{Nf} \mathbf{S}_f \\ s_{Pf} w_{Nf} \mathbf{S}_f^T & -g_f \end{pmatrix}, \quad (7)$$

with $\gamma_P = \sum_f s_{Pf} w_{Pf} \mathbf{S}_f$ plus boundary terms, δ_P the corresponding divergence row (equal to γ_P on internal faces), a_{PN} the upwind-plus-diffusion coupling, and $[\boldsymbol{\Omega}]_\times$ the cross-product matrix. The pressure-gradient column of the momentum rows and the velocity-divergence row are transposes of each other; the (3, 3) entries are the Rhie–Chow stabilisation of the otherwise saddle-point system. The blocks are stored row-major, 16 values per cell and per face (Section 4.1); rows 0–2 are momentum, row 3 is continuity.

Continuity-row scaling. The continuity row is multiplied by

$$s_p = \frac{1}{\max(\langle D \rangle, \varepsilon_v)}, \quad \langle D \rangle = \text{global mean of } D_P, \quad (8)$$

before the system is narrowed to single precision, so that the magnitudes of the continuity and momentum rows are comparable. Since this is a pure row scaling, the solution vector needs no unscaling; the residual norms of Section 3.2 are those of the scaled system.

2.5 Boundary conditions

The specification tabulated the implicit treatment of a small set of boundary-condition pairs, and the table was partly lost in its source (D-002). The implementation therefore treats every physical patch uniformly through the linearisation that every OpenFOAM patch field provides,

$$\mathbf{U}_b = \alpha_b^U \odot \mathbf{U}_P + \beta_b^U, \quad p_b = \alpha_b^p p_P + \beta_b^p, \quad (9)$$

with α, β from `valueInternalCoeffs(w)` and `valueBoundaryCoeffs(w)` (componentwise product $\odot$). The momentum rows take the native `internalCoeffs/boundaryCoeffs` of the upwind convection and uncorrected Laplacian operators, and the pressure gradient contributes $\mathbf{S}_f \alpha_b^p$ to column 3 of the diagonal block and $-\mathbf{S}_f \beta_b^p$ to the right-hand side. The continuity row takes $\mathbf{S}_f^T \text{diag}(\alpha_b^U)$ in columns 0–2, $-\mathbf{S}_f \cdot \beta_b^U$ on the right-hand side, and on fixed-pressure faces the Rhie–Chow term $-g_b(p_b - p_P)$ linearised with (9). Because the linearisation is native, `fixedValue`, `zeroGradient`, `slip/symmetry`, `inletOutlet`, `totalPressure` and any condition derived from fixed-value, zero-gradient or mixed patches work without special code (the mixed value fraction of `inletOutlet` and the dynamic head of `totalPressure` are evaluated from the previous outer iteration). Wall functions act through ν_t on the wall face. For outflow through a fixed-value face the native convection boundary term is kept (D-019).

Processor patches are fully implicit: their coefficients are stored as block interface coefficients and applied during the matrix–vector product (Section 5). Cyclic, cyclicAMI and processor-cyclic patches are explicitly coupled in this version: neighbour values are lagged from the previous outer iteration and enter the right-hand side; implicit block coupling across them is left to Phase 2.

For closed domains (p needs a reference), the continuity row of the reference cell is replaced by $d(p_{\text{ref}} - p_{\text{value}}) = 0$, where d is its former diagonal (D-021). This is exact, because the continuity equations of a closed incompressible domain sum to zero and one of them is redundant. The native weak reference, which doubles the diagonal of the reference cell, leaves a near-null space; once the right-hand side is rounded to single precision its compatibility condition is slightly violated and the float Krylov solve drifts along the constant-pressure mode. This was observed on the cavity case and removed by the strong reference.

2.6 Implicit MRF

The native MRF formulation solves for the absolute velocity, convects it with the relative flux, and adds $\boldsymbol{\Omega} \times \mathbf{U}$ as an explicit source; this equals the relative-frame Coriolis and centrifugal terms $2\boldsymbol{\Omega} \times \mathbf{U}_r + \boldsymbol{\Omega} \times (\boldsymbol{\Omega} \times \mathbf{r})$. `coupledFoam` makes the same term implicit by adding $V_P[\boldsymbol{\Omega}]_\times$ to the 3×3 momentum part of the diagonal block (D-006; a factor 2 would be wrong for the absolute velocity). The per-cell matrix is extracted from the native operator `MRFZoneList::DDt` applied to the three unit vector fields, so any number of zones and time-dependent $\boldsymbol{\Omega}$ are handled through the public API only; it is re-extracted every outer iteration. As in `simpleFoam`, the flux is stored relative (`MRF.makeRelative` after the flux update), because the turbulence model convects with the same flux object (D-014, addendum).

2.7 Non-orthogonal correction with a per-face limiter

For a face-normal gradient the full face value is $\partial_n \psi|_f = \Delta_f(\psi_N - \psi_P) + c_f(\psi)$. The correction c_f is the native `limited corrected` λ correction: the explicit non-orthogonal part of the `corrected` scheme, limited so that its magnitude does not exceed $\lambda/(1 - \lambda)$ times the orthogonal part. Two limiters are evaluated, the global `nonOrthLimiter` (default 0.5) and the static-remediation limiter (default 0.2); faces adjacent to a static-remediation cell take the latter (Section 3.8). The same correction is used for the momentum Laplacian, where it enters as the explicit source $\nabla \cdot (\nu_f |\mathbf{S}_f| c_f(\mathbf{U}))$ (zero on physical boundary faces), and for the Rhie–Chow pressure Laplacian through q_f in (6). On an exactly orthogonal mesh, detected from the native correction vectors, both are skipped.

2.8 Turbulence coupling

The turbulence model is used unmodified through OpenFOAM’s incompressible turbulence-model interface (e.g. k - ω SST [21], GEKO [22]); `turbulence->correct()` is called once per outer iteration after the coupled update, with the native solvers and relaxation factors for k and ω . Afterwards $k \geq k_{\min}$ and $\omega \geq \omega_{\min}$ are enforced and ν_t is capped at $10^5 \nu$ (the number of capped cells is logged).

3 Solution algorithm

3.1 Increment formulation and the double-precision residual

Let $\mathcal{A}(\mathbf{x})$ be the block operator assembled at the state $\mathbf{x} = (\mathbf{U}, p)$ (convection linearised on the lagged flux, explicit terms evaluated at $\mathbf{x}$) and $\mathbf{b}(\mathbf{x})$ its right-hand side. Assembly is in double precision. The residual

$$\mathbf{r} = \mathbf{S}(\mathbf{b} - \mathcal{A}\mathbf{x}), \quad \mathbf{S} = \text{diag}(1, 1, 1, s_p) \text{ per cell}, \quad (10)$$

is accumulated in double from the double coefficients *before* the coefficients are narrowed to single precision (D-012). The single-precision matrix $\mathbf{A}_f = \text{fl}_{32}(\mathbf{S}\mathcal{A})$ is used only to compute an increment,

$$\mathbf{A}_f \delta \mathbf{x} \approx \text{fl}_{32}(\mathbf{r}), \quad \mathbf{x} \leftarrow \mathbf{x} + \omega \delta \mathbf{x}, \quad (11)$$

with the line-search factor ω of Section 3.6. This is a mixed-precision iterative refinement [3, 4] superposed on the nonlinear (Picard/defect-correction) iteration: the single-precision matrix, the inexact Krylov solve and the pseudo-time term change only the convergence rate, while the fixed point is that of the double residual. On the cavity test this is what allows $R < 10^{-8}$ with a float linear solver. The narrowing clamps values with $|v| > 10^{30}$ to $\pm 10^{30}$ and counts them; a non-zero count is a warning and a test failure.

Because the pseudo-time term is added with the current iterate as the old value, it contributes $(V_P/\Delta t_P) \delta \mathbf{U}_P$ to the matrix and nothing to $\mathbf{r}$. In increment form the row scaling and the continuity row need no special treatment.

3.2 Residual norms

With $\mathbf{r}$ and $\mathcal{A}\mathbf{x}$ of the scaled system, the solver uses

$$\nu_n = \sum_i (|(\mathcal{A}\mathbf{x})_i| + |b_i|) + \epsilon, \quad R_n = \frac{\|\mathbf{r}_n\|_2 / \nu_{n,n}}{\|\mathbf{r}_1\|_2 / \nu_{n,1}}, \quad (12)$$

summed over all four components of all cells, the analogue of OpenFOAM’s `normFactor`. R_n is the combined residual that drives the CFL control and the stop criterion (D-016). For comparison with native solvers the log also reports the L_1 -normalised residuals r_U (rows 0–2) and

r_p (row 3), $\sum |r_i| / \sum (|(\mathcal{A}\mathbf{x})_i| + |b_i|)$, which are directly comparable with the “initial residual” of `simpleFoam`.

3.3 Outer iteration

Algorithm 1 summarises one outer iteration. Assembly is split into two stages, because the solution-limited time step (Section 3.5) needs the momentum residual before the pseudo-time term is known, whereas $D = V/\bar{a}$ needs the pseudo-time term.

Algorithm 1. Outer iteration n of `coupledFoam`.

$\nu_{\text{eff}}; \beta_P$ (start-up, remediation, zonal); CFL factors f_P
repeat (at most $1 + \text{maxCflCuts}$ times)
 $V_P/\Delta t_P \leftarrow \lambda_P/(\text{CFL } f_P)$ (local time step, eq. 13)
 assemble momentum rows; momentum residual $\mathbf{r}_P^U$ without pseudo-time term
 apply the solution-limited cap to $V_P/\Delta t_P$ (eq. 17)
 add $V_P/\Delta t_P; \bar{a}_P, D_P, D_f, q_f$; assemble continuity row; reference cell
 narrow to float; scale row 3; $\mathbf{r} = \mathbf{S}(\mathbf{b} - \mathcal{A}\mathbf{x})$ in double; R_n
 solve $\mathbf{A}_f \delta \mathbf{x} = \mathbf{r}$ (FGMRES + block GAMG, rel. tolerance η_n)
 if the solve failed ($\eta_n \|\mathbf{r}_0\|$ and absolute floor not reached in `maxIter`, or non-finite):
 if `maxLinFails` consecutive failures: diagnostic exit (Section 3.9)
 else: $\omega \leftarrow$ physicality line search (eq. 18)
 if ($\omega < \omega_{\min}$ or failed) and cuts $< \text{maxCflCuts}$:
 $\text{CFL} \leftarrow \max(\text{CFL}_{\min}, \kappa \text{CFL})$; **repeat**
if failed: skip the update; $\text{CFL} \leftarrow \max(\text{CFL}_{\min}, 0.25 \text{CFL})$ (only if `maxCflCuts` $< \text{maxLinFails}$)
else if $\omega < \omega_{\min}$: $\omega \leftarrow \omega_{\min}$; mark offending cells dynamic
 clip increments of dynamic cells; store rollback copy; $\mathbf{U} += \omega \delta \mathbf{U}, p += \omega \delta p$
 update boundary values; ϕ_f from eq. 5; make ϕ relative in MRF zones
 sentinel check, rollback on failure (Section 3.9)
 dynamic remediation update; `turbulence->correct()`; bounds; sentinel check
 CFL strategy update with R_n ; start-up switch; log; write; convergence test

3.4 Pseudo-transient continuation and local time step

The local pseudo-time step combines a convective and a diffusive limit,

$$\frac{V_P}{\Delta t_P} = \frac{\max(\lambda_P, \varepsilon_v)}{\max(\text{CFL } f_P, \varepsilon_v)}, \quad \lambda_P = \frac{1}{2} \sum_f |\phi_f| + \nu_{\text{eff},P} V_P^{1/3}, \quad (13)$$

with the global adaptive CFL number and the product $f_P \leq 1$ of the remediation and zonal CFL factors (Section 3.8). The global CFL follows one of three strategies [5, 6, 23]:

$$\text{mRDM (default): } \text{CFL}_{n+1} = \begin{cases} \min(\text{CFL}_{\max}, \text{CFL}_n \max(1, \min(\beta_{\max}, (R_{n-1}/R_n)^\gamma))) & R_n \leq R_{n-1}, \\ \text{CFL}_n & \text{otherwise,} \end{cases} \quad (14)$$

$$\text{EXP: } \text{CFL}_{n+1} = \min(\text{CFL}_{\max}, \beta_{\text{exp}} \text{CFL}_n), \quad (15)$$

$$\text{SER: } \text{CFL}_{n+1} = \min(\text{CFL}_{\max}, \max(\text{CFL}_{\min}, \text{CFL}_n R_{n-1}/R_n)), \quad (16)$$

with defaults $\gamma = 1$, $\beta_{\max} = 1.5$, $\beta_{\text{exp}} = 1.1$, $\text{CFL}_0 = 5$, $\text{CFL}_{\min} = 1$, $\text{CFL}_{\max} = 500$. mRDM and EXP never decrease the CFL number; decreases come only from the line search, failed solves and the sentinel. After every decrease a hold of $n_{\text{hold}} = 10$ iterations forbids increases. With the linear-solve failure trigger of amendment B4 the recovery after a decrease is governed by mRDM (bounded by $\beta_{\max}$), not by a fixed ramp; after `maxLinFails` (3) consecutive failed linear solves the run takes the diagnostic exit of Section 3.9.

3.5 Solution-limited local time step

The momentum residual of the current assembly (without the pseudo-time term, but including the pressure gradient) estimates the local change a step would cause, $\delta U_P = |\mathbf{r}_P^U| \Delta t_P / V_P$. If $\delta U_P > f_{\text{loc}} U_{\text{ref}}$ ($f_{\text{loc}} = 0.5$), the cell's time step alone is reduced,

$$\frac{V_P}{\Delta t_P} \leftarrow \frac{V_P}{\Delta t_P} \frac{\delta U_P}{f_{\text{loc}} U_{\text{ref}}}, \quad (17)$$

so that the global CFL can stay aggressive while individual cells are throttled; the number of limited cells is logged.

3.6 Physicality line search

Given the increment, the step length is

$$\omega = \min\left(1, \min_P \frac{f_U U_{\text{ref}}}{\max(|\delta \mathbf{U}_P|, \varepsilon_v)}, \min_P \frac{f_P p_{\text{ref}}}{\max(|\delta p_P|, \varepsilon_v)}\right), \quad (18)$$

reduced over all ranks, with $f_U = 0.3$, $f_P = 0.5$, $U_{\text{ref}} = \max |\mathbf{U}|$ (internal and physical boundary values) at the first iteration and $p_{\text{ref}} = \frac{1}{2} U_{\text{ref}}^2$, both frozen and stored for restart [6]. If $\omega < \omega_{\text{min}} = 0.1$, the CFL number is cut by $\kappa = 0.5$ and the step is repeated from the same fields (reassembly with the new Δt), at most three times; then the step is accepted with $\omega = \omega_{\text{min}}$ and every cell whose increment would have violated the bounds at ω_{min} is marked into the dynamic remediation set.

A linear solve that does not reach its target $\eta_n \|\mathbf{r}_{\text{lin},0}\|$ (or the absolute tolerance floor) within **maxIter** iterations, or that produces non-finite values, is a failed step (amendment B4, D-029, which replaces the earlier “no residual reduction” rule of D-020): near the float floor a Krylov solve can diverge, and applying even ω_{min} of such an increment destroys a converged state. A failed solve cuts the CFL number by κ and repeats the outer step from unchanged fields; the repeat counts toward **maxCflCuts**. After **maxLinFails** (3) consecutive failures the run leaves through the diagnostic path of Section 3.9. The fallback of D-020 (skip the step without a field update and cut the CFL number by the sentinel factor) remains only for settings with **maxCflCuts** < **maxLinFails**. In addition, block BiCGStab returns the iterate with the smallest residual if it does not converge, so its output is never worse than the initial guess (D-020, item 1); the GMRES variants minimise the residual over their Krylov space by construction.

3.7 Start-up

Fresh runs are initialised from **potentialFoam** (called by the case script with **coupled.potentialInit yes**); in the benchmark its wall and CPU time are counted as part of the coupledFoam solution (Section 7). For the first **startupUpwindIters** (50) iterations, or until $R_n < 10^{-2}$, $\beta_P = 0$ everywhere, i.e. the discretisation is pure upwind; afterwards $\beta_P = 1$ except in remediated cells. On restart the start-up phase is skipped.

3.8 Remediation cell sets

Static set. Built once from OpenFOAM's per-face mesh-quality measures: a cell is marked if any of its faces has non-orthogonality above 70° or skewness above 4, if its volume ratio to any neighbour exceeds 20, or its aspect ratio exceeds 1000. Static cells receive $\beta_P = \min(\beta_P, \beta_{\text{static}})$ with $\beta_{\text{static}} = 0$ (upwind) by default, the stronger non-orthogonal limiter 0.2 on their faces, a limited (**cellLimited Gauss linear 1**) pressure gradient in the Rhie–Chow term, and a CFL factor 0.5. The gradient inside the user's high-order convection scheme cannot be overridden per cell; with the default $\beta_{\text{static}} = 0$ it does not enter (D-018).

Dynamic set. Updated every iteration. A cell is marked if $|\mathbf{U}_P| > 2U_{\text{ref}}$, $|p_P| > 5p_{\text{ref}}$, $|\mathbf{U}_P - \bar{\mathbf{U}}_{N(P)}| > 0.5 U_{\text{ref}}$ (face-neighbour mean), any of $\mathbf{U}, p, k, \omega$ is non-finite, or it is passed in by the line search or the sentinel. Marked cells plus one layer of face neighbours (across processor boundaries) receive $\beta_P = 0$, a CFL factor 0.1 and increment clipping,

$$|\mathbf{U}_P + \omega \delta \mathbf{U}_P - \bar{\mathbf{U}}_{N(P)}| \leq c_{\text{spike}} U_{\text{ref}}, \quad c_{\text{spike}} = 0.5, \quad (19)$$

which rescales the increment and never overwrites the field. A cell leaves the set after 20 consecutive iterations without being re-marked. If the union of both sets exceeds 1 % of the cells a warning is issued.

Zonal factors. Optionally (amendment B6), multipliers on the local time step and on β_P are applied in named cell zones and within n_{layers} face-neighbour layers of named patches (layers computed once by a breadth-first search from the patch faces, across processor faces); several matching entries multiply. The effective blending factor and CFL factor are

$$\beta_P = z_P^\beta \cdot \begin{cases} 0 & P \text{ dynamic,} \\ \min(\beta_{\text{glob}}, \beta_{\text{static}}) & P \text{ static,} \\ \beta_{\text{glob}} & \text{otherwise,} \end{cases} \quad f_P = z_P^{\text{CFL}} \min(1, f_P^{\text{static}}, f_P^{\text{dyn}}), \quad (20)$$

with $\beta_{\text{glob}} \in \{0, 1\}$ from the start-up switch and $f_P^{\text{static}}, f_P^{\text{dyn}}$ equal to one outside the respective sets. Local pseudo-time is implicit local under-relaxation; explicit per-patch relaxation factors are deliberately not provided, because they would break the increment consistency of the line search, and processor boundaries need no relaxation since the block interfaces are fully implicit.

3.9 Sentinel, rollback and failure handling

Before every field update, $\mathbf{U}, p, \phi$ and (if present) k, ω, ν_t are copied. After the update and again after the turbulence correction a single pass computes the extrema of $p, |\mathbf{U}|, k, \omega$ and the number of non-finite values. The check fails if any value is non-finite, $\max |\mathbf{U}| > 10 U_{\text{ref}}$ or $\max |p| > 50 p_{\text{ref}}$. On failure the copies are restored, the CFL number is cut to $\max(\text{CFL}_{\text{min}}, 0.25 \text{ CFL})$, the offending cells are marked into the dynamic set and a rollback is counted. After five consecutive rollbacks the restored fields are written to a time directory `<iter>_lastValid` together with the remediation flag and a `sentinelFlag` field of the offending cells, their centres are printed, and the run stops with a fatal error. A time directory containing non-finite values is never written.

3.10 Restart

At every write time the native fields $(\mathbf{U}, p, \phi, k, \omega, \nu_t)$ are written, followed by the dictionary `coupledState`: iteration, CFL, previous R , hold counter, start-up flag, $R_1, U_{\text{ref}}, p_{\text{ref}}$, dynamic set and ages, sentinel counters, and a placeholder for an AMR refinement history. The field D of the last assembly is written alongside. `coupledState` is written to a temporary file and renamed atomically, last, so its presence marks a complete time directory (D-017). On restart the following are restored: the fields $\mathbf{U}, p, \phi$ and the turbulence fields; the PTC state (iteration, CFL, previous R , hold counter, R_1 , start-up flag, $U_{\text{ref}}, p_{\text{ref}}$); the dynamic set and its ages; the sentinel counters; the Eisenstat–Walker state; the state of the cycle-efficiency controller (current cycle type, post-smoothing count, window and hysteresis); and the window of the force-coefficient convergence monitor. Deliberately not restored is the Anderson history (Section 3.12): a restarted run starts with an empty history (D-026), so a run with Anderson acceleration is reproducible across a restart only up to that history. ϕ is recomputed from the read $\mathbf{U}, p$ with the stored D by (5); the maximum relative difference to the read flux is logged as the flux-consistency metric (required below 10^{-6} in the restart test).

3.11 Eisenstat–Walker forcing

With `adaptiveRelTol` (default), the relative tolerance of the linear solve in outer iteration n is the forcing term of Eisenstat and Walker’s Choice 2 [11],

$$\eta_n = \gamma_{\text{EW}} \left(\frac{R_n}{R_{n-1}} \right)^{\alpha_{\text{EW}}}, \quad \gamma_{\text{EW}} = 0.9, \alpha_{\text{EW}} = 2, \quad (21)$$

with the safeguards, in this order: (i) if $\gamma_{\text{EW}} \eta_{n-1}^{\alpha_{\text{EW}}} > 0.1$ then $\eta_n \leftarrow \max(\eta_n, \gamma_{\text{EW}} \eta_{n-1}^{\alpha_{\text{EW}}})$; (ii) $\eta_n \leftarrow \min(\eta_{\text{max}}, \max(\eta_{\text{min}}, \eta_n))$ with $\eta_{\text{min}} = 10^{-3}$, $\eta_{\text{max}} = 0.5$; (iii) during start-up and (iv) at $n = 1$, $\eta_n = \eta_{\text{max}}$. The absolute tolerance remains a floor and at least one Krylov iteration is always performed, so the increment is never zero. The minimum-iteration rule matters: if the absolute linear tolerance is reached before the outer target, a zero increment freezes the outer iteration (observed on the cavity: R frozen at 7.0×10^{-8} , D-022).

3.12 Anderson acceleration

Optionally (default off), the accepted outer update is accelerated by Anderson mixing [2] in the Type II form of Walker and Ni [37]. The outer iteration is read as the fixed-point map $\mathbf{x} \mapsto G(\mathbf{x}) = \mathbf{x} + \mathbf{f}(\mathbf{x})$ with $\mathbf{f} = \omega \delta \mathbf{x}$ the accepted (line-searched, clipped) increment. With the last m differences $\Delta \mathbf{x}_j = \mathbf{x}_{j+1} - \mathbf{x}_j$ and $\Delta \mathbf{f}_j = \mathbf{f}_{j+1} - \mathbf{f}_j$,

$$\gamma = \arg \min_{\gamma} \left\| \mathbf{W} \left(\mathbf{f}_k - \sum_j \gamma_j \Delta \mathbf{f}_j \right) \right\|_2, \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \beta \mathbf{f}_k - \sum_j \gamma_j (\Delta \mathbf{x}_j + \beta \Delta \mathbf{f}_j), \quad (22)$$

where $\mathbf{W}$ scales velocities by $1/U_{\text{ref}}$ and pressure by $1/p_{\text{ref}}$; for $\beta = 1$ this is $\mathbf{x}_{k+1} = G(\mathbf{x}_k) - \sum_j \gamma_j \Delta G_j$. The least-squares problem is solved with a thin QR factorisation updated column by column (classical Gram–Schmidt with one re-orthogonalisation, all dot products in double and reduced over ranks; the oldest column is removed by Givens rotations); a numerically dependent new column is not added. Safeguards: the update is skipped (history kept) if $\max_j |\gamma_j| > 10$; the extrapolated state must pass the sentinel, otherwise the un-extrapolated state is restored and the history flushed. The history is further flushed on every CFL cut and every skipped step, when the line search returns the minimum step ω_{min} , on every rollback, whenever the dynamic remediation set changes (detected by a version counter of the set, not by its size), at the start-up switch (β from 0 to 1, which changes the discretisation), and whenever the cycle-efficiency controller changes the preconditioner (which changes the inner solve and hence the map). Convergence theory for contractive and non-contractive operators and for Picard iterations of the Navier–Stokes equations is given in [28, 29]. The memory cost is $(2m + 2)$ double 4-vectors per cell, 320 B/cell for $m = 4$ (14.4 GB at 45 M cells, Table 1), which is why the option is off by default and not used above about 35 M cells.

4 Linear algebra

4.1 Storage and precision

The specification asked for OpenFOAM’s mixed-precision build (SPDP) with single-precision `solveScalar`. In OpenFOAM v2606 the opposite holds: SPDP stores fields in float and solves in double, and no build option gives float solve with double fields (D-001). OpenFOAM is therefore built in double precision, and the library defines its own types: `blockScalar = float` for block coefficients and all Krylov and multigrid vectors, and `reduceScalar = double` for every accumulation (dot products, norms, Galerkin sums) and every MPI reduction. The native `solveScalar` is never used for block data, so the library is correct under DP, SPDP and SP builds, and the native reference solver runs with the same double-precision fields.

Table 1: Memory budget at 45 M cells, worst case (amendment B7). The 45 M-cell run itself is a user acceptance step outside the one-shot scope; Figure 41 compares the measured peak RSS of the benchmark cases with this budget scaled per cell. The Anderson row uses the implemented 320 B/cell (D-026); the 11.5 GB of the B7 table assumed 256 B/cell.

item	approx. GB
mesh, fields, addressing (double)	40–45
block matrix 4×4 (single)	20
block-GAMG hierarchy, $C_{\text{op}} \leq 1.5$ (single)	20–30
FGMRES, restart 6: 12 block vectors (single)	8.6
K-cycle workspace, all levels	5
turbulence, rollback copies, reserve	10
total	105–120
Anderson, $m = 4$ (optional)	+14.4

The block matrix uses OpenFOAM’s LDU addressing: 16 floats per cell for the diagonal blocks (row-major, index $16c + 4r + j$), 16 per internal face for the upper (owner row, neighbour column) and 16 for the lower blocks, 4 per cell for vectors, and 16 per face for each processor interface. On a hexahedral mesh with about three internal faces per cell this is about $64 + 2 \cdot 3 \cdot 64 = 448$ B per cell, half of the double-precision equivalent. Table 1 gives the budget for the largest target mesh.

Double-precision reductions. All inner products and norms go through a small `doubleReduce` layer that accumulates float products in double locally and reduces with `MPI_DOUBLE`; the multi-value variants reduce several sums in one call to save latency. A float sum over 10^7 – 10^8 entries loses three to four digits and $\|\cdot\|^2$ can overflow. The unit test sums 10^8 copies of $\text{fl}_{32}(10^{-4})$: the double accumulator reproduces $10^8 \cdot \text{fl}_{32}(10^{-4})$ exactly (the exact value 9999.99974737875 differs from 10^4 because 10^{-4} is not representable in float, D-004), whereas the float accumulator is wrong by more than 1 %.

Dense 4×4 kernels. Block inversion uses LU with partial pivoting in double; a pivot with magnitude below 10^{-30} is shifted by 10^{-30} and counted, so a singular block never produces NaN or Inf. Block products that feed stored results accumulate in double; the matrix–vector kernels inside Krylov and smoother loops run in float.

4.2 Krylov solvers

All solvers act on the $4N$ -vector with the block matrix–vector product (native LDU traversal, processor exchange posted before and completed after the local work) and test convergence on $\|\mathbf{r}\|_2/\nu_n$ with the outer normalisation ν_n supplied by the assembler: converged if the value is below the absolute tolerance or below η_n times its initial value, after at least one iteration.

blockBiCGStab. Right-preconditioned BiCGStab (Saad [32], Alg. 7.7, arranged as the native `PBiCGStab`). Breakdown guards $|\rho| < \varepsilon_v \|\mathbf{r}_0\|^2$ or $|\omega| < \varepsilon_v$ restart from the current iterate with a fresh shadow residual (at most three times). Best-iterate return as in Section 3.6.

blockGMRES. Restarted GMRES(m) with right preconditioning (Alg. 9.5): Arnoldi on $\mathbf{A}\mathbf{M}^{-1}$, only the basis $\mathbf{V}$ is stored and $\mathbf{x} += \mathbf{M}^{-1}(\mathbf{V}\mathbf{y})$ is formed once per cycle ($m + 3$ block vectors). Correct only for a fixed preconditioner.

blockFGMRES. Flexible GMRES(m) [31] (Alg. 9.6): the preconditioned vectors $\mathbf{z}_j = \mathbf{M}_j^{-1}\mathbf{v}_j$ are stored, so a preconditioner that changes between applications (the K-cycle, an iterative

Table 2: Cycle work per finest-level work unit, equation (23) (infinitely many levels). The K-cycle lies between V and W depending on how often the second GCR step is taken.

coarsening ratio r	2	3	4	8
V	2.00	1.50	1.33	1.14
F	4.00	2.25	1.78	1.31
W	∞	3.00	2.00	1.33

coarsest-level solve) is handled correctly; $2m + 2$ block vectors, $m = 10$ by default and 6 above 40 M cells. Arnoldi uses modified Gram–Schmidt with double-accumulated inner products; the Hessenberg matrix, the Givens rotations and the least-squares solve are in double. A happy breakdown $h_{j+1,j} < \varepsilon_v \|\mathbf{r}_0\|$ ends the cycle.

The combination of the K-cycle with a solver other than blockFGMRES is rejected at start-up.

4.3 Block additive-correction AMG

Hierarchy. The agglomeration is OpenFOAM’s own `GAMGAgglomeration` (default `faceAreaPair`, which uses geometry only), obtained on the mesh and cached; with `mergeLevels = 2` two pairwise agglomeration steps are merged into one level. Restriction $\mathbf{R}_l$ sums the fine residual blocks of each coarse cell, prolongation $\mathbf{P}_l = \mathbf{R}_l^T$ is piecewise constant, and the coarse operator is the Galerkin product $\mathbf{A}_{l+1} = \mathbf{R}_l \mathbf{A}_l \mathbf{P}_l$ evaluated by summation [14, 38]: coarse diagonal blocks collect the fine diagonal blocks of their cells and both off-diagonal blocks of every fine face interior to the coarse cell; coarse face blocks collect the fine face blocks with the native face-flip map; processor interface blocks are summed with the native patch-face restriction. Sums are in double, results are stored in float. Every level uses the same block matrix class on the native coarse `lduPrimitiveMesh`.

Operator complexity and coarsening ratio. With $C_{\text{op}} = \sum_l \text{nnz}_l / \text{nnz}_0$ (in blocks) and the measured global ratios $r_l = N_l / N_{l+1}$, an agglomeration is accepted if $C_{\text{op}} \leq 1.5$ and, for W- and K-cycles, every $r_l \geq 3$; otherwise `mergeLevels` is increased (up to 4) and the mesh re-agglomerated, and if the rule still fails the run stops with the measured values. The rule follows from the work of one cycle per finest-level work unit for a constant ratio r :

$$W_V = \sum_{l \geq 0} r^{-l} = \frac{r}{r-1}, \quad W_F = \sum_{l \geq 0} (l+1) r^{-l} = \left(\frac{r}{r-1} \right)^2, \quad W_W = \sum_{l \geq 0} \left(\frac{2}{r} \right)^l = \frac{r}{r-2} \quad (r > 2), \quad (23)$$

see Table 2. With plain pairwise agglomeration ($r = 2$) the W-cycle, and a K-cycle that takes its second step on every level, cost $O(LN)$ instead of $O(N)$.

Smoothers. Block Gauss–Seidel, $\mathbf{x}_P \leftarrow \mathbf{D}_P^{-1}(\mathbf{b}_P - \sum_N \mathbf{A}_{PN} \mathbf{x}_N)$, with precomputed dense 4×4 inverses and the loop structure of the native `GaussSeidelSmoother`; processor contributions are Jacobi-like (values of the previous sweep). Block ILU(0) is implemented in the LDU-native form, the block analogue of the native DILU preconditioner (D-015):

$$\mathbf{M} = (\mathbf{D}^* + \mathbf{L}) \mathbf{D}^{*-1} (\mathbf{D}^* + \mathbf{U}), \quad \mathbf{D}_u^* = \mathbf{D}_u - \sum_{f: u(f)=u} \mathbf{L}_f \mathbf{D}_{l(f)}^{*-1} \mathbf{U}_f, \quad (24)$$

with one sweep $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{M}^{-1}(\mathbf{b} - \mathbf{A}\mathbf{x})$. Only diagonal blocks are modified, so the factorisation stays inside the LDU pattern; full ILU(0) would also update couplings between two neighbours of a common cell, which are not faces and hence not stored. Processor interfaces enter the ILU(0) sweep only through the residual.

Cycles. Level 0 is the finest, L the coarsest, $\mathcal{S}_l(\mathbf{x}, \mathbf{b}, \nu)$ denotes ν smoothing sweeps ($\nu_{\text{pre}} = 1$, $\nu_{\text{post}} = 2$, and $\nu = 2$ for both on the finest level). Algorithm 2 gives the cycles; the F-cycle does one W-style recursion per level followed by a V-cycle on the remaining residual. The K-cycle of Notay and Vassilevski [25], which underlies AGMG [24], replaces the coarse correction by one or two steps of a generalised conjugate residual (GCR) method preconditioned by the cycle on the next level (Algorithm 3); the second step is taken only if the first reduced the coarse residual by less than the factor $t = 0.25$. The cycle thus behaves like a V-cycle where the coarse correction is effective and like a W-cycle where it is not, per level and per application. Additive-correction aggregation AMG has a comparatively weak coarse-grid correction, which is why a plain V-cycle does not scale optimally with it. Because the K-cycle is a nonlinear, variable preconditioner, it requires the flexible outer Krylov method.

Algorithm 2. CYCLE($l, \mathbf{b}$): approximate solution of $A_l \mathbf{x} = \mathbf{b}$ from $\mathbf{x} = 0$.

```

if  $l = L$ : return coarsest solve of  $A_L \mathbf{x} = \mathbf{b}$  (dense LU or BiCGStab)
 $\mathbf{x} \leftarrow \mathcal{S}_l(0, \mathbf{b}, \nu_{\text{pre}})$ ;  $\mathbf{r} \leftarrow \mathbf{b} - A_l \mathbf{x}$ ;  $\mathbf{b}_c \leftarrow R_l \mathbf{r}$ 
V:  $\mathbf{e}_c \leftarrow \text{CYCLE}(l+1, \mathbf{b}_c)$ 
F:  $\mathbf{e}_c \leftarrow \text{CYCLE}_F(l+1, \mathbf{b}_c)$ ;  $\mathbf{e}_c \leftarrow \text{CYCLE}_V(l+1, \mathbf{b}_c - A_{l+1} \mathbf{e}_c)$ 
W:  $\mathbf{e}_c \leftarrow \text{CYCLE}(l+1, \mathbf{b}_c)$ ;  $\mathbf{e}_c \leftarrow \text{CYCLE}(l+1, \mathbf{b}_c - A_{l+1} \mathbf{e}_c)$ 
K:  $\mathbf{e}_c \leftarrow \text{KSTEP}(l+1, \mathbf{b}_c)$ 
 $\mathbf{x} \leftarrow \mathbf{x} + P_l \mathbf{e}_c$ ;  $\mathbf{x} \leftarrow \mathcal{S}_l(\mathbf{x}, \mathbf{b}, \nu_{\text{post}})$ ; return  $\mathbf{x}$ 

```

Algorithm 3. KSTEP($l, \mathbf{b}$): Notay–Vassilevski K-step, GCR with at most $k_{\text{max}} = 2$ steps; all inner products double-accumulated.

```

 $\mathbf{r}_0 \leftarrow \mathbf{b}$ 
 $\mathbf{z}_1 \leftarrow \text{CYCLE}(l, \mathbf{r}_0)$ ;  $\mathbf{q}_1 \leftarrow A_l \mathbf{z}_1$ 
 $\alpha_1 \leftarrow \langle \mathbf{q}_1, \mathbf{r}_0 \rangle / \langle \mathbf{q}_1, \mathbf{q}_1 \rangle$ ;  $\mathbf{e} \leftarrow \alpha_1 \mathbf{z}_1$ ;  $\mathbf{r}_1 \leftarrow \mathbf{r}_0 - \alpha_1 \mathbf{q}_1$ 
if  $\|\mathbf{r}_1\| \leq t \|\mathbf{r}_0\|$  or  $k_{\text{max}} = 1$ : return  $\mathbf{e}$ 
 $\mathbf{z}_2 \leftarrow \text{CYCLE}(l, \mathbf{r}_1)$ ;  $\mathbf{q}_2 \leftarrow A_l \mathbf{z}_2$ 
 $\theta \leftarrow \langle \mathbf{q}_2, \mathbf{q}_1 \rangle / \langle \mathbf{q}_1, \mathbf{q}_1 \rangle$  (with the unorthogonalised  $\mathbf{q}_2$ )
 $\mathbf{q}_2 \leftarrow \mathbf{q}_2 - \theta \mathbf{q}_1$ ;  $\mathbf{z}_2 \leftarrow \mathbf{z}_2 - \theta \mathbf{z}_1$ 
 $\alpha_2 \leftarrow \langle \mathbf{q}_2, \mathbf{r}_1 \rangle / \langle \mathbf{q}_2, \mathbf{q}_2 \rangle$ ;  $\mathbf{e} \leftarrow \mathbf{e} + \alpha_2 \mathbf{z}_2$ ; return  $\mathbf{e}$ 

```

The K-step workspace ($\mathbf{z}_1, \mathbf{q}_1, \mathbf{z}_2, \mathbf{q}_2, \mathbf{r}$) is allocated once per level and cached. On the finest level the Krylov acceleration is provided by the outer FGMRES, so K-steps are applied on the coarse levels only.

Coarsest level. If the coarsest level resides on one rank with at most 64 cells, it is solved by a dense LU factorisation in double of the assembled $(4n) \times (4n)$ matrix, refactorised only when the matrix changes. Otherwise block BiCGStab with the block-diagonal preconditioner is used to a relative tolerance of 10^{-3} or 50 iterations. The selection is automatic and logged.

Processor agglomeration. On many ranks the cost of coarse levels is latency, not arithmetic. Coarse levels are gathered onto fewer ranks with OpenFOAM’s **GAMGProcAgglomeration** (**masterCoarsest**): a level with fewer than $5000 n_{\text{ranks}}$ cells is agglomerated onto $\max(1, n_{\text{ranks}}/4)$ ranks, and a level below 5000 cells onto one rank. Block interface coefficients follow the native mapping with 16 coefficients per face instead of one. Skipping coarse levels is not used: it removes the low-frequency correction and saves less than 5 % of the cycle work.

Cycle-efficiency controller. The preconditioner efficiency $\rho = \|\mathbf{b} - A M^{-1} \mathbf{b}\| / \|\mathbf{b}\|$ is measured on the first preconditioner application of every linear solve (in FGMRES it comes from the

first Arnoldi step at the cost of one extra reduction). Every 50 outer iterations the median ρ_{med} of the window is evaluated: if $\rho_{\text{med}} > 0.7$ the post-smoothing count is increased (up to 4) and, at the maximum, V is promoted to F and F to W (refused if the coarsening-ratio rule does not hold); a K-cycle is not promoted further (it adapts internally). If $\rho_{\text{med}} < 0.3$ the post-smoothing count is decreased, and a cycle promoted by the controller is demoted by one step once the count is back at one sweep and $\rho_{\text{med}} < 0.2$. Every change requires the condition in two consecutive windows, is applied between linear solves only, and is logged as an event; $\rho_{\text{med}} > 0.9$ for two windows at the maximum setting with a W- or K-cycle is reported as a failure.

5 Implementation in OpenFOAM

Library structure. `coupledFoam` is an add-on library `libcoupledFoam` and an application built with `wmake` into the user’s library and application directories; OpenFOAM itself is not modified. The library has five parts: `blockMatrix` (precision types, 4×4 kernels, double reductions, the block LDU matrix and processor interfaces), `blockSolvers` (Krylov solvers, block-diagonal and block-GAMG preconditioners, smoothers), `assembly` (the two-stage assembler, deferred-correction convection, Rhie–Chow interpolation, limited non-orthogonal correction, boundary classification and implicit MRF), `control` (PTC and CFL control, line search, remediation, sentinel, convergence monitor, Anderson acceleration, GAMG controller) and `io` (restart state, run information, JSON output). Project code is compiled with `-Wall -Wextra -Werror` (OpenFOAM headers as system headers) and `-O3 -march=native -fno-math-errno; -ffast-math` is excluded because it breaks NaN and Inf detection.

Runtime selection and settings. Solvers, preconditioners and smoothers are selected at run time from the `coupled` entry of `fvSolution.solvers`; the outer-iteration controls live in the `coupled` sub-dictionary of `fvSolution`. Every constant is a dictionary entry with a documented default; the solver prints the complete effective settings at start-up, a single machine-parseable CFI line per iteration (CFL, ω , cuts, R , r_U , r_p , linear iterations and residual, assembly/solve/turbulence times, set sizes, rollbacks, clamps) and a JSON summary at the end (iterations, wall and CPU time summed over ranks, peak RSS, GAMG levels and C_{op}).

Parallel interfaces. Each processor patch is wrapped by a block interface that exchanges 4-vectors per face with non-blocking MPI, using the native neighbour rank, tag, communicator and face ordering, on the finest level and on the native coarse GAMG interfaces. Unlike the native `interfaceBouCoeffs`, whose product is subtracted, the block coefficients are stored as the actual matrix entries A_{PN} of the remote neighbour. Processor-cyclic, cyclic and cyclicAMI interfaces are not block-coupled (Section 2.5).

Floating-point safety. Development and test runs trap floating-point exceptions (FOAM_SIGFPE, FOAM_SETNAN); any trap is a test failure. Benchmark runs disable traps and enable flush-to-zero and denormals-are-zero. Every division by a computed quantity, every pivot and every argument of `sqrt`, `log` and `pow` is guarded; the guards carry a common source tag so that they can be reviewed. Narrowing to float clamps and counts, and the sentinel prevents non-finite states from propagating or being written.

Implementation status. The baseline (Sections 2 to 3.10, block BiCGStab/GMRES with the V-cycle block GAMG) was completed first; the components of amendment B (FGMRES, Eisenstat–Walker forcing, F/W/K cycles with the coarsening-ratio rule, dense coarsest LU, processor agglomeration, the cycle controller, Anderson acceleration and zonal factors) were integrated afterwards. Every run records its effective settings, and the results in Sections 6 and 7 are reported with the configuration that produced them.

Testing strategy. Three layers, all driven by `pytest` and writing one JSON record per test: (i) unit-test applications on the real mesh classes: `Test-precision` (type sizes), `Test-block40ps` (1000 random inversions, $\|AA^{-1} - I\|_{\infty} < 10^{-5}$, guarded singular block), `Test-blockMatrix` (block product against native `lduMatrix::Amul` componentwise, 1 and 4 ranks), `Test-doubleReduce`, `Test-blockGAMG` (block Poisson system, iteration count and 1-vs-4-rank identity, and the comparison of the V-, F-, W- and K-cycles, Figure 45) and `Test-blockFGMRES` (convergence with a deliberately variable preconditioner, where standard GMRES may stall); (ii) case tests T0–T5, T-restart, T-fpe and the informational T-mrf, each against `simpleFoam` on the identical mesh and schemes; (iii) the benchmark harness of Section 7. Test runs enable floating-point traps and record the machine load, since the development machine is shared (D-007).

6 Verification and validation

6.1 Test cases

Unless stated otherwise, every case is run on 1 and on 4 ranks (scotch decomposition); both must pass, with a cross-rank tolerance of 10^{-4} relative on integral quantities. Native references are produced with `simpleFoam` on the identical mesh, schemes and boundary conditions.

T0, lid-driven cavity (laminar). 128×128 cells, $Re = 100$ and 1000, `linearUpwind` convection. Pass: $R < 10^{-8}$ within 300 iterations; centreline profiles $u(y)$ and $v(x)$ (129 points) within 10^{-3} relative L_2 of `simpleFoam`; no floating-point trap; no clamped coefficients. Ghia et al. [13] provide the classical reference data.

T1, pitzDaily (SST). About 12 k cells. Pass: $R < 10^{-6}$ within 400 iterations; inlet-to-outlet pressure drop within 1 % of `simpleFoam`; $CFL \geq 100$ without cuts after iteration 100; no rollbacks.

T2, backward-facing step 2D (SST). Pass: reattachment length (first sign change of the lower-wall shear stress downstream of the step) within 2 % of `simpleFoam`; at least $2 \times$ fewer outer iterations to 10^{-5} (definitions of the two residuals in D-024). Experimental reference: Driver and Seegmiller [10].

T3, airFoil2D (SST and GEKO). Pass: C_l, C_d within 0.5 % of `simpleFoam` for each model with identical solver settings; empty dynamic set at convergence.

T4, motorBike. `snappyHexMesh` meshes of about 350 k cells (a) and 1–2 M cells (b; retargeted from the specified 3–5 M cells, D-031), 10 ranks (the development machine is capped at 10 cores for this work, D-031). Each mesh is built once and reused by all tests and benchmarks. Pass: both solvers reach a stationary window mean of C_d and C_l (averaged force criterion for oscillating wakes, D-042, Section 7); window means within $\max(2\%, 0.002)$ (C_d) and $\max(2\%, 0.01)$ (C_l) of `simpleFoam`; RMS of the mean-field differences at most 0.02 (proposed, D-042 addendum); static set at most 1 % of the cells; no rollbacks.

T5, Ahmed body, 25° slant. Generated geometry, half model, 5–8 M cells, $U_{\infty} = 40$ m/s, SST with wall functions. Pass: as T4 (averaged force criterion, D-042); comparison with the experimental $C_d = 0.285$ [1] ($\pm 10\%$) is informational.

T-restart. T1 and T3-SST, split at iteration 150: iteration count ± 2 , final quantities within 10^{-5} , flux consistency below 10^{-6} .

T-fpe. T1 from $\mathbf{U} = 0$, no potential initialisation, $CFL_0 = 200$, no upwind start-up: no trap, recovery (rollbacks allowed) and convergence to $R < 10^{-5}$.

Table 3: Test results (spec 13).

test	pass	iterations	final R	wall [s]	CPU-h	key metric
T0_Re1000_np1	yes	111	9.69e-09	26	0.00715	l2rel_u = 1.43e-05
T0_Re1000_np4	yes	257	5.88e-09	70.8	0.079	l2rel_u = 1.41e-05
T0_Re100_np1	yes	57	4.25e-09	5.74	0.00171	l2rel_u = 2.89e-06
T0_Re100_np4	yes	57	6.7e-09	5.72	0.00692	l2rel_u = 2.89e-06
T1_np1	yes	400	2.75e-06	28.1	0.00779	dpRelDiff = 5.72e-04
T1_np4	yes	400	5.7e-06	17.5	0.0188	dpRelDiff = 5.73e-04
T2_np1	yes	1000	8.89e-06	228	0.0632	xrRelDiff = 6.85e-03
T2_np4	yes	1000	9.06e-06	90.4	0.1	xrRelDiff = 6.43e-03
T3_GEKO_np1	NO	3000	0.000231	822	0.228	CdRelDiff = 3.87e-02
T3_kOmegaSST_np1	NO	3000	0.0149	684	0.19	CdRelDiff = 1.16e+00
Test-block4Ops	yes	n/a	n/a	n/a	n/a	maxInverseError = 6.24e-08
Test-blockFGMRES	yes	n/a	n/a	n/a	n/a	
Test-blockGAMG	yes	n/a	n/a	n/a	n/a	crossRankRelDiff = 4.48e-06
Test-blockGAMG_cycles	NO		n/a		n/a	
Test-blockGAMG_np1	yes	n/a	n/a	0.132	n/a	
Test-blockGAMG_np4	yes	n/a	n/a	0.136	n/a	
Test-blockMatrix	yes	n/a	n/a	n/a	n/a	crossRankRelDiff = 1.71e-09
Test-blockMatrix_np1	yes	n/a	n/a	n/a	n/a	
Test-blockMatrix_np4	yes	n/a	n/a	n/a	n/a	
Test-doubleReduce	yes	n/a	n/a	n/a	n/a	
Test-doubleReduce_np1	yes	n/a	n/a	n/a	n/a	relErrorDouble = 0.00e+00
Test-doubleReduce_np4	yes	n/a	n/a	n/a	n/a	relErrorDouble = 0.00e+00
Test-precision	yes	n/a	n/a	n/a	n/a	
test_env	yes	n/a	n/a	n/a	n/a	

T-mrf. mixerVessel2D (MRF, SST), torque against `simpleFoam`; informational, since no threshold is specified (D-009).

T-scaling. Strong scaling on T4b with 1, 2, 4, 8 and 10 ranks instead of the specified 1–16 (10-core cap, D-031; recorded as `specRanks=false`). Pass: the parallel efficiency of coupledFoam at the largest rank count is at least 80 % of that of `simpleFoam` (unchanged threshold; Figure 44).

6.2 Results

Table 3 lists all test records produced so far.

T0, $Re = 100$. coupledFoam converged to $R < 10^{-8}$ in 57 outer iterations; the reference `simpleFoam` (SIMPLEC) run needed 1932 iterations to its residual tolerance of 10^{-8} (the two residual normalisations differ, Section 3.2). The centreline profiles agree to a relative L_2 difference of 2.9×10^{-6} in u and 5.5×10^{-6} in v (Figure 1), far below the pass threshold of 10^{-3} and consistent with two converged solutions of the same discretisation. The history (Figure 2) shows the CFL ramp and the line-search factor. The corresponding profiles at $Re = 1000$ are shown in Figure 3; the test record is listed in Table 3. The wall times of the test records were measured on a shared machine and are not benchmark results.

T1–T5. Integral quantities and their differences to the native solver are collected in Table 4; remediation activity on the snappyHexMesh cases is shown in Figure 4.

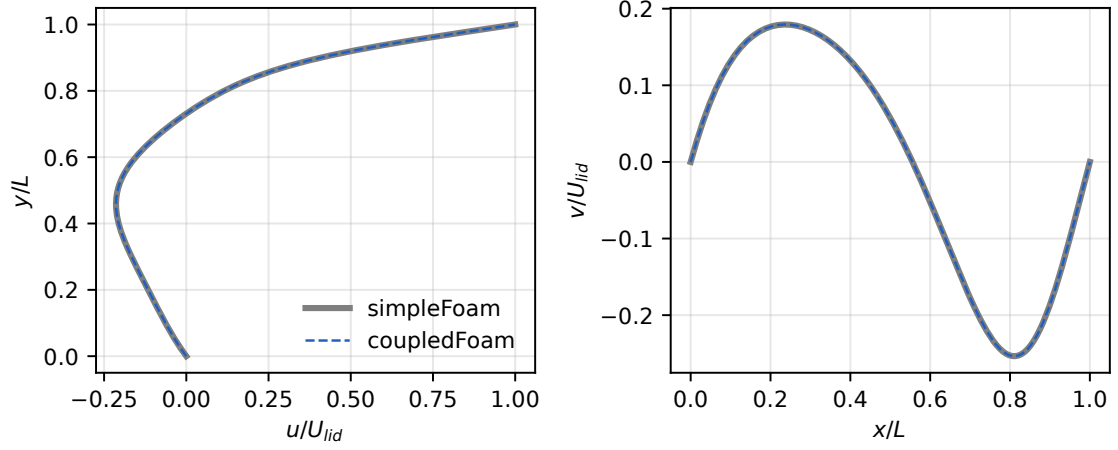


Figure 1: T0, lid-driven cavity at $Re = 100$: centreline profiles $u(y)$ and $v(x)$, coupledFoam against simpleFoam.

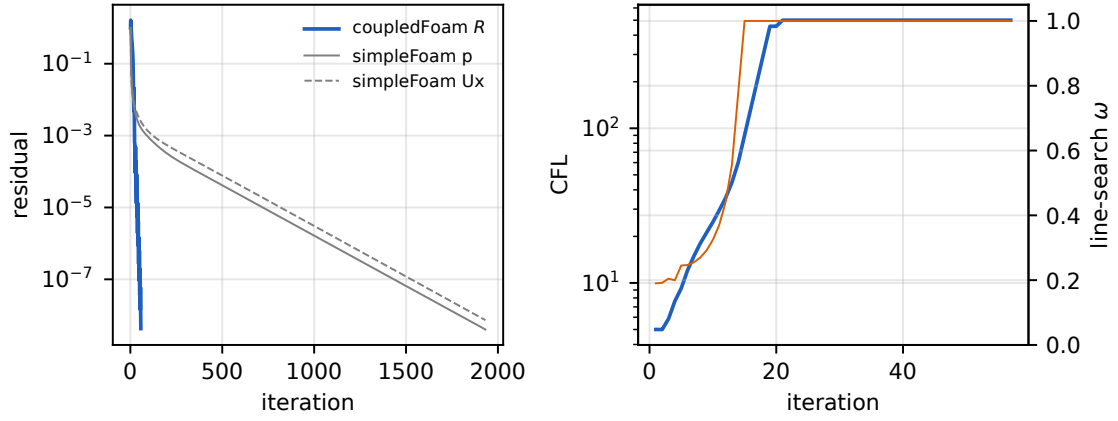


Figure 2: T0, $Re = 100$, one rank: combined residual R of coupledFoam and native initial residuals of simpleFoam (left); CFL number and line-search factor ω (right).

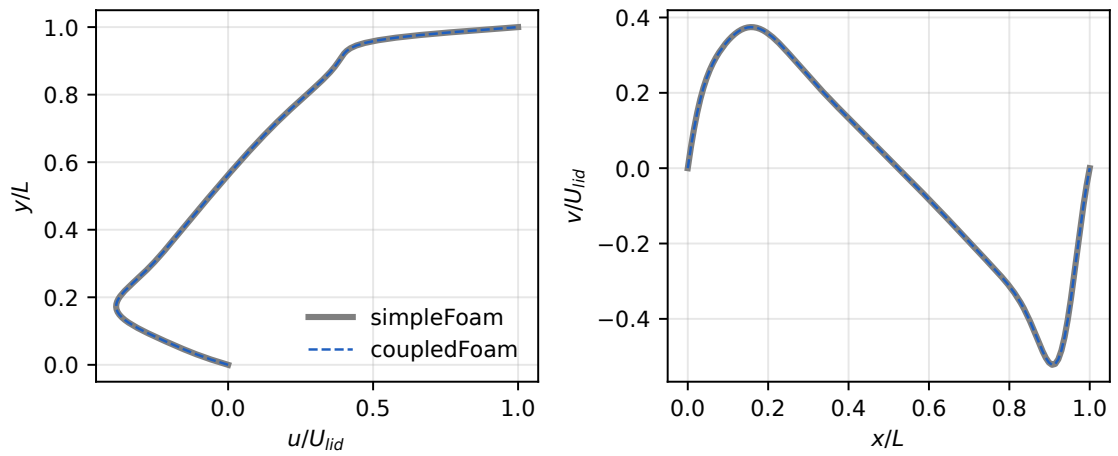


Figure 3: T0, lid-driven cavity at $Re = 1000$: centreline profiles, coupledFoam against simpleFoam.

Table 4: Validation: integral quantities, coupledFoam vs. simpleFoam on identical meshes and schemes; wake cases T4 and T5 compared by window means and mean-field deltas (D-042 addendum).

case	quantity	coupledFoam	simpleFoam	rel. diff.	tolerance	field RMS dU / dp	pass
T1_np1	dp	-5.7287	-5.7254	5.72e-04	0.01		yes
T2_np1	x_r/h	6.4348	6.391	6.85e-03	0.02		yes
T3_kOmegaSST_np1	Cd	-0.014659	0.090727	1.16e+00	0.005		NO
T3_kOmegaSST_np1	Cl	-0.45645	0.25267	2.81e+00	0.005		NO
T3_GEKO_np1	Cd	0.032969	0.034296	3.87e-02	0.005		NO
T3_GEKO_np1	Cl	0.91365	0.83781	9.05e-02	0.005		NO

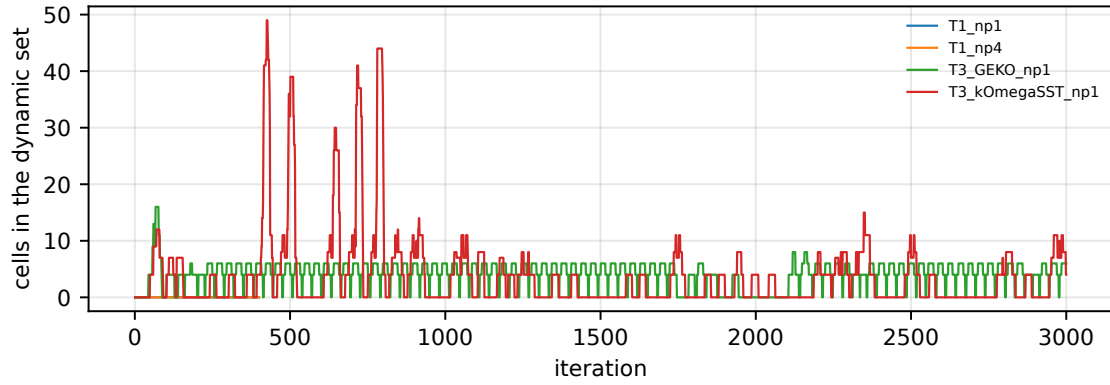


Figure 4: Remediation set sizes and rollbacks over the iterations for T4 and T5.

T0 lid-driven cavity, Re 100: velocity magnitude

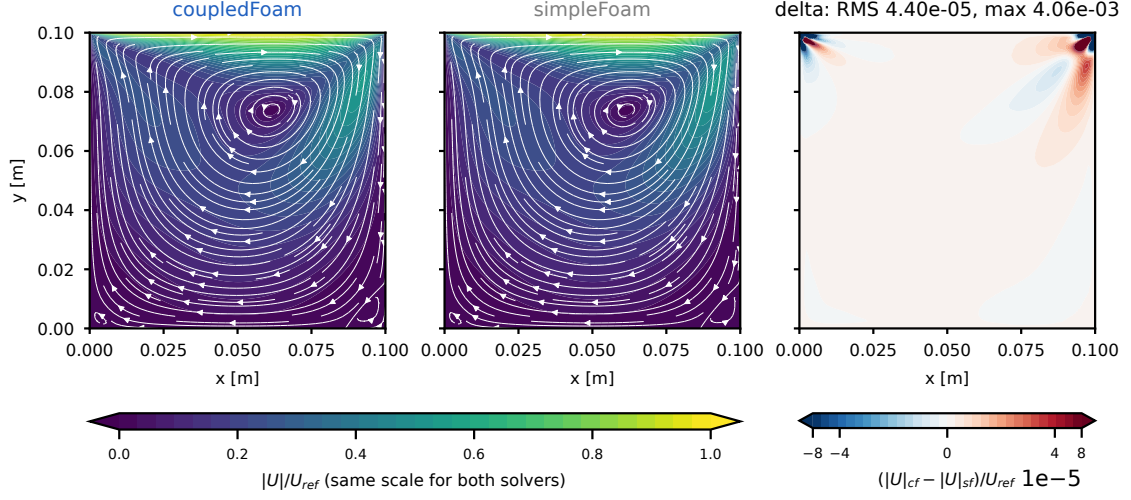


Figure 5: T0, $Re = 100$: $|\mathbf{U}|/U_{lid}$ with streamlines, coupledFoam (left) and simpleFoam (middle) on one colour scale, and the difference (right).

6.3 Flow fields

Integral quantities can agree through a compensation of errors, so the converged fields of the two solvers are also compared directly. For every 2D case with a coupledFoam run and a simpleFoam reference on the same mesh, the figures of this section show the velocity magnitude $|\mathbf{U}|/U_{ref}$ with streamlines and the pressure coefficient $C_p = (p - p_0)/(\frac{1}{2}U_{ref}^2)$ of both solvers on one colour scale, and the pointwise difference coupledFoam–simpleFoam on a diverging colour scale. The fields are read from the latest time directory of each run with the OpenFOAM reader of VTK, interpolated to the mesh points and cut at mid-depth (`bench/plot_fields2d.py`); the meshes are identical, so the difference is evaluated node by node, and the quoted RMS is weighted with the area. U_{ref} is the lid speed (T0), the inlet speed (T1: 10 m/s, T2: 44.2 m/s) or the free-stream speed (T3: 26 m/s); p_0 is the outlet pressure, except in the closed cavity, where the area-weighted mean pressure is removed from both fields.

T0. The velocity differences are at the level of the converged profile differences: RMS 4.4e-05 at $Re = 100$ and 3.6e-05 at $Re = 1000$ (Figures 5–8). The largest differences sit in the two upper corners, where the moving lid meets the walls: the pressure is singular there and both solutions are least resolved.

T0 lid-driven cavity, Re 100: pressure coefficient

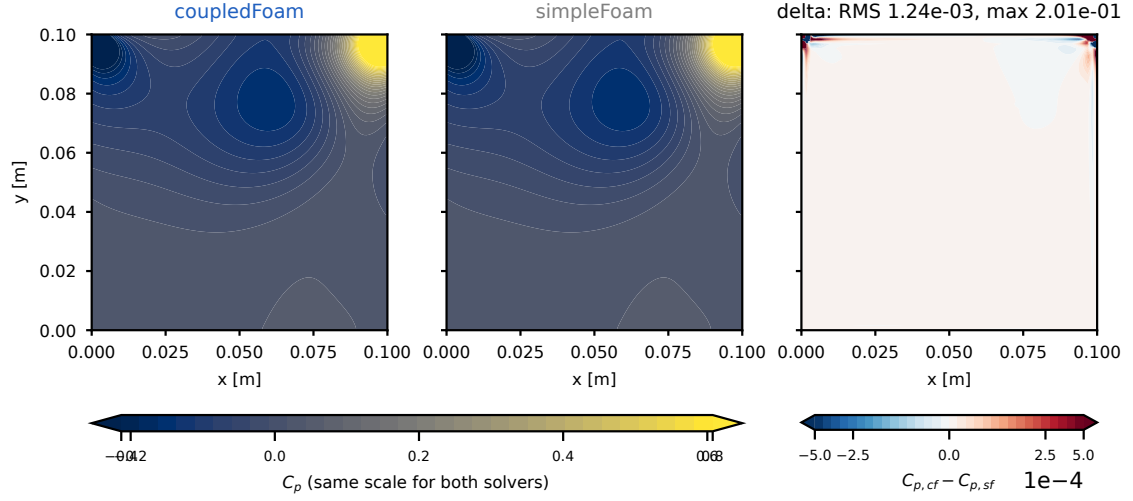


Figure 6: T0, $Re = 100$: pressure coefficient (mean removed) of both solvers and the difference.

T0 lid-driven cavity, Re 1000: velocity magnitude

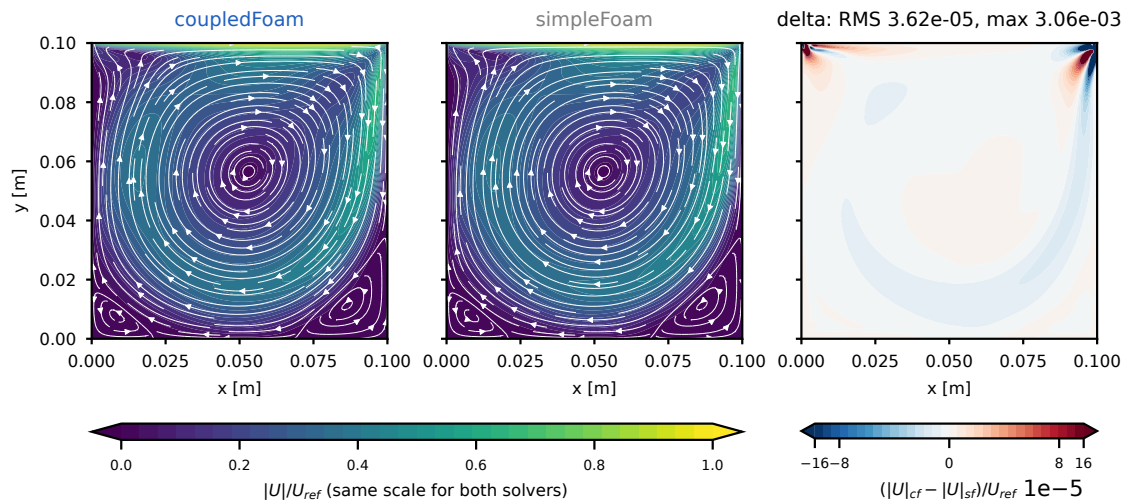


Figure 7: T0, $Re = 1000$: $|U|/U_{lid}$ with streamlines and the difference.

T0 lid-driven cavity, Re 1000: pressure coefficient

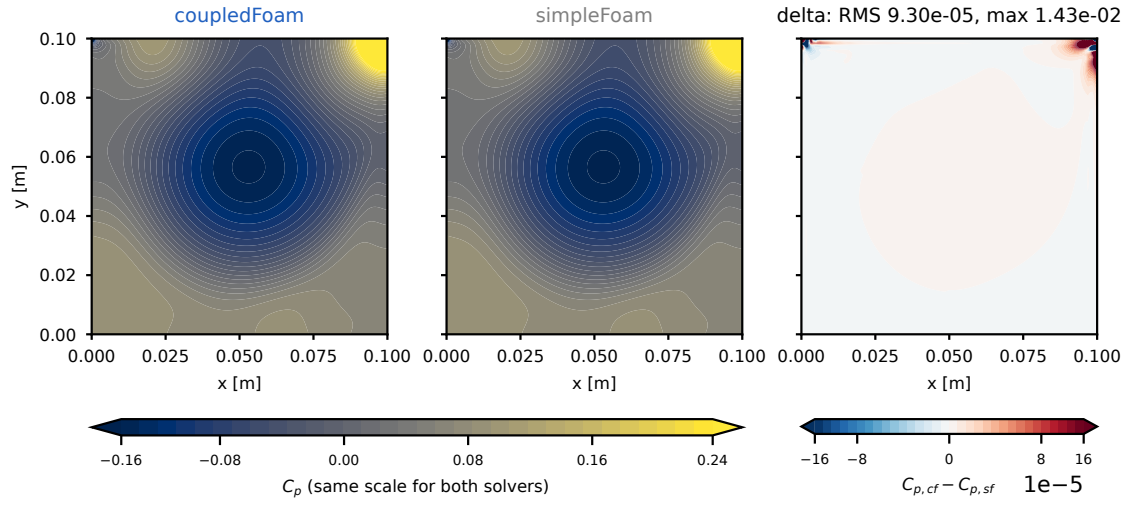


Figure 8: T0, $Re = 1000$: pressure coefficient and the difference.

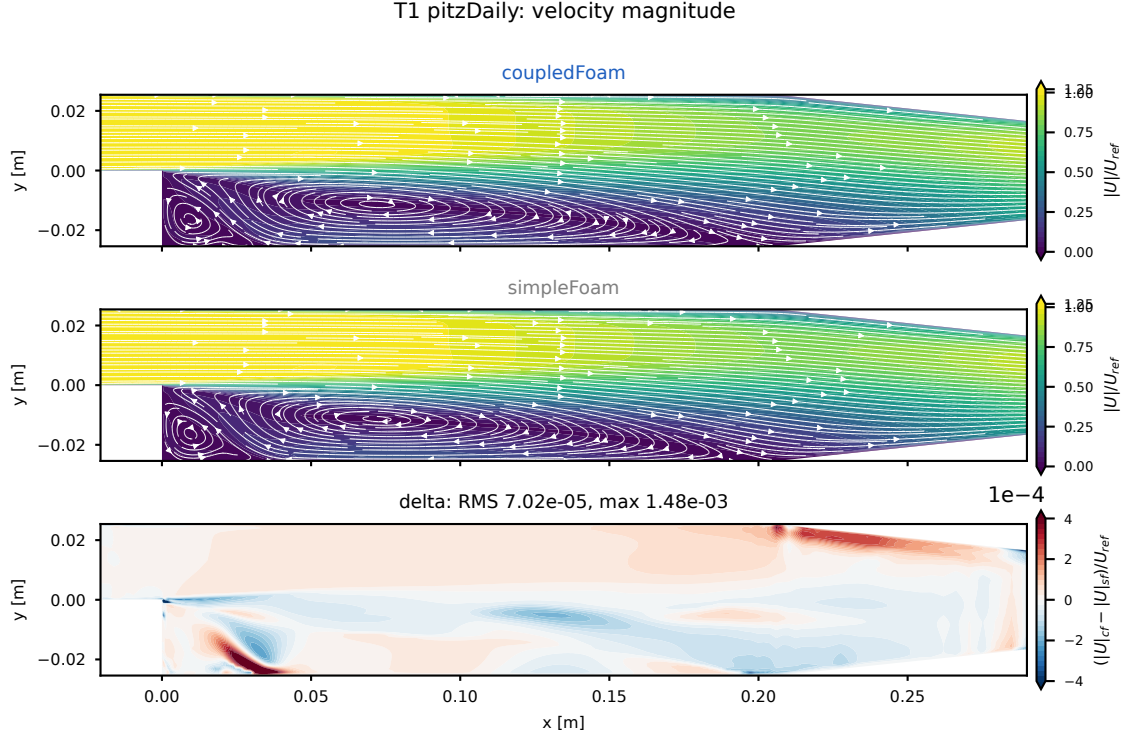


Figure 9: T1 pitzDaily: $|\mathbf{U}|/U_{ref}$ with streamlines, coupledFoam (top), simpleFoam (middle), difference (bottom).

T1. The recirculation behind the step, its length and the velocity profiles coincide (Figures 9–11); the RMS differences are $7.0e-05$ in $|\mathbf{U}|/U_{ref}$ and $1.4e-04$ in C_p , consistent with the pressure-drop difference of Table 4.

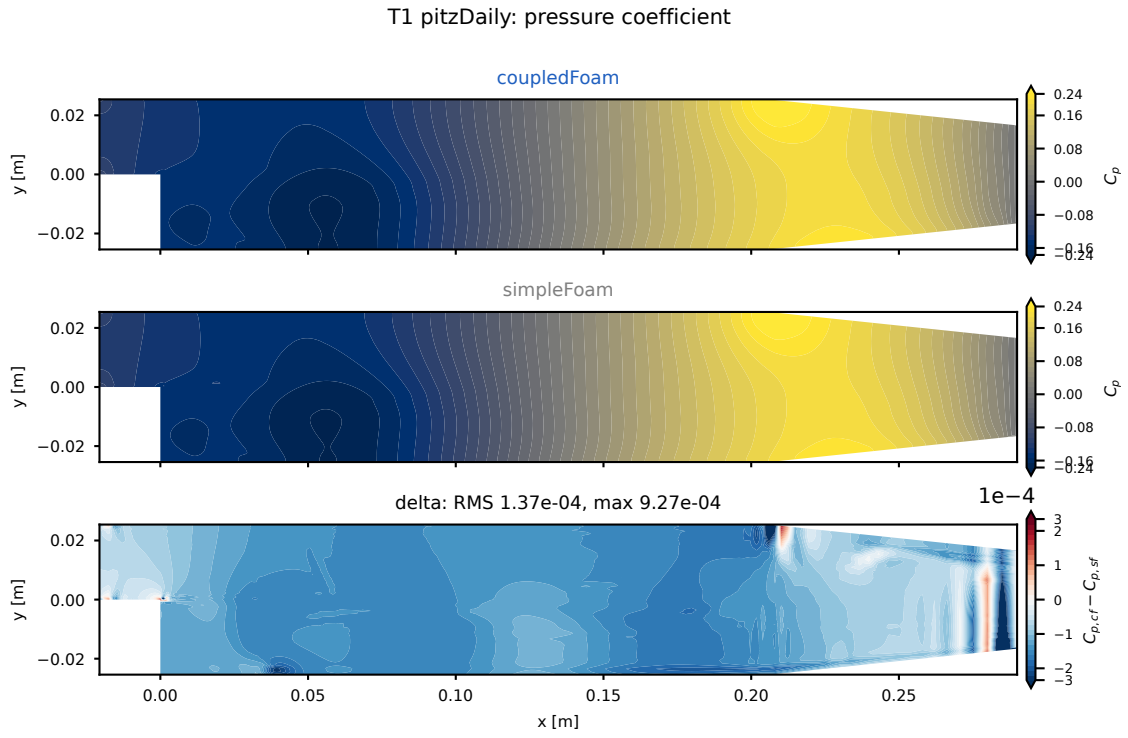


Figure 10: T1 pitzDaily: pressure coefficient and the difference.

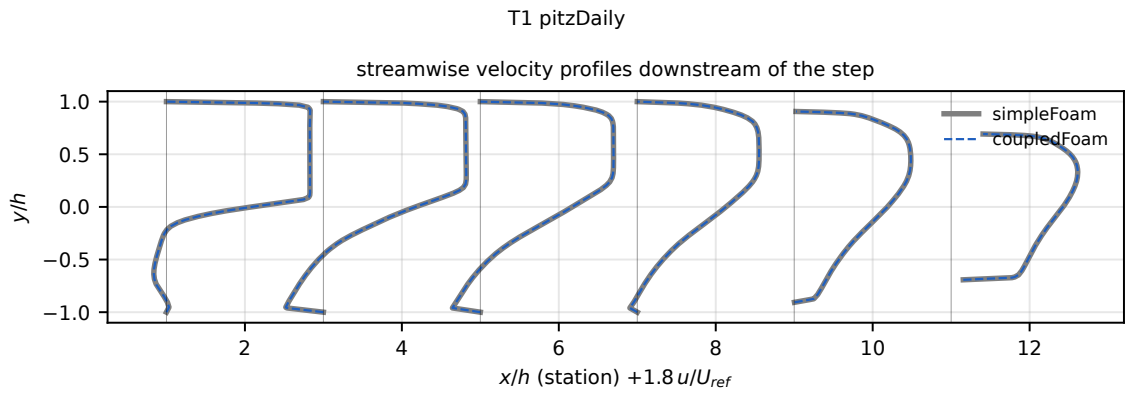


Figure 11: T1 pitzDaily: streamwise velocity profiles at the stations x/h downstream of the step (h step height), simpleFoam thick grey, coupledFoam dashed blue.

T2. Separation at the step corner, the recirculation bubble and the recovery agree (Figures 12–14); the RMS difference in $|\mathbf{U}|/U_{ref}$ is 1.1e-03, the maximum (1.5e-01) sits in the thin shear layer leaving the step corner. The lower-wall skin friction crosses zero at the same point, which is the reattachment-length test. The `simpleFoam` skin friction has a narrow spike near $x/h \approx 1.4$ that `coupledFoam` does not have; the `simpleFoam` run did not reach the residual target in 20 000 iterations (Table 3), so the spike is not interpreted further.

T2 backward-facing step: velocity magnitude

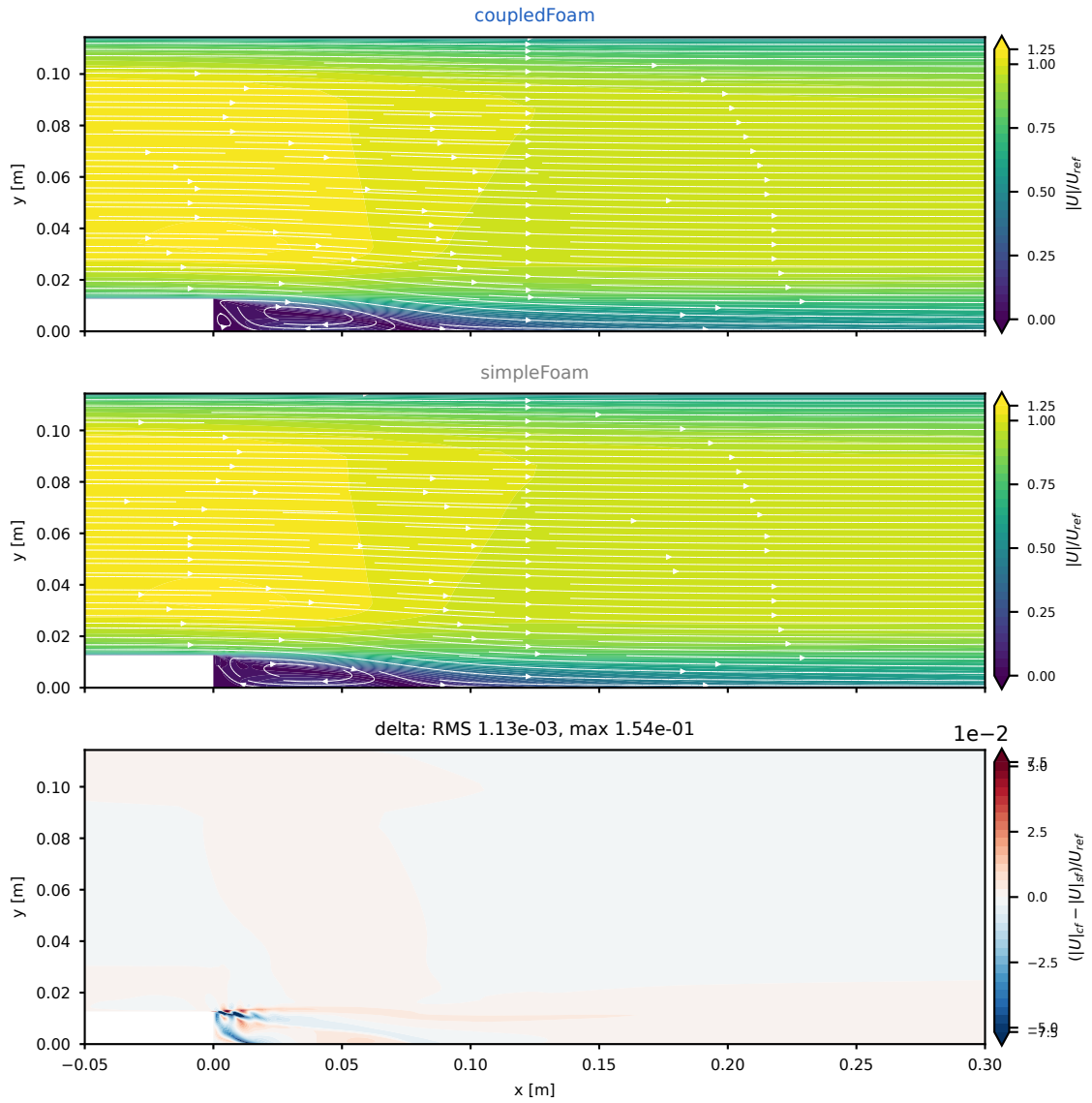


Figure 12: T2 backward-facing step: $|\mathbf{U}|/U_{ref}$ with streamlines, coupledFoam (top), simpleFoam (middle), difference (bottom).

T2 backward-facing step: pressure coefficient

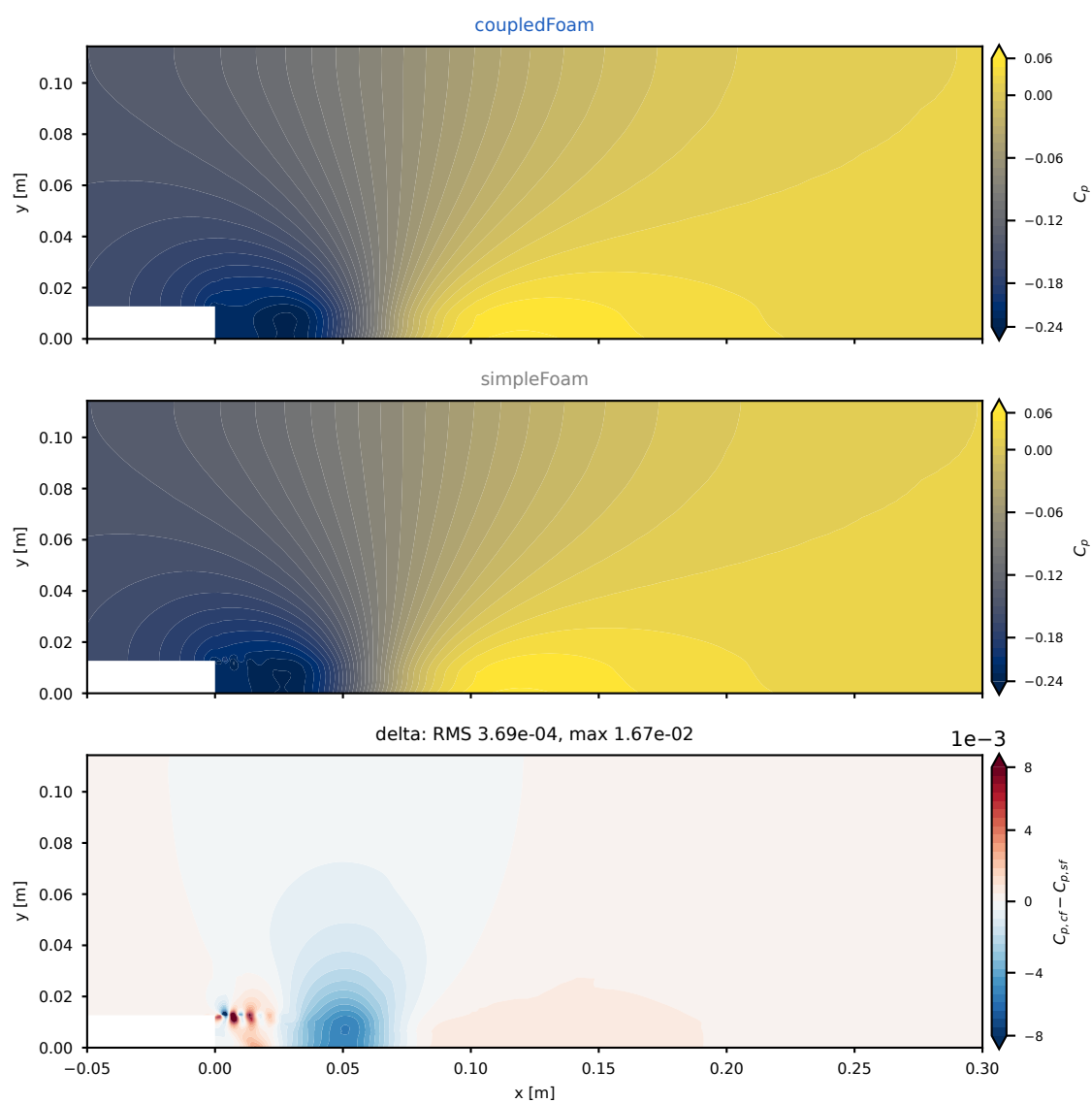


Figure 13: T2 backward-facing step: pressure coefficient and the difference.

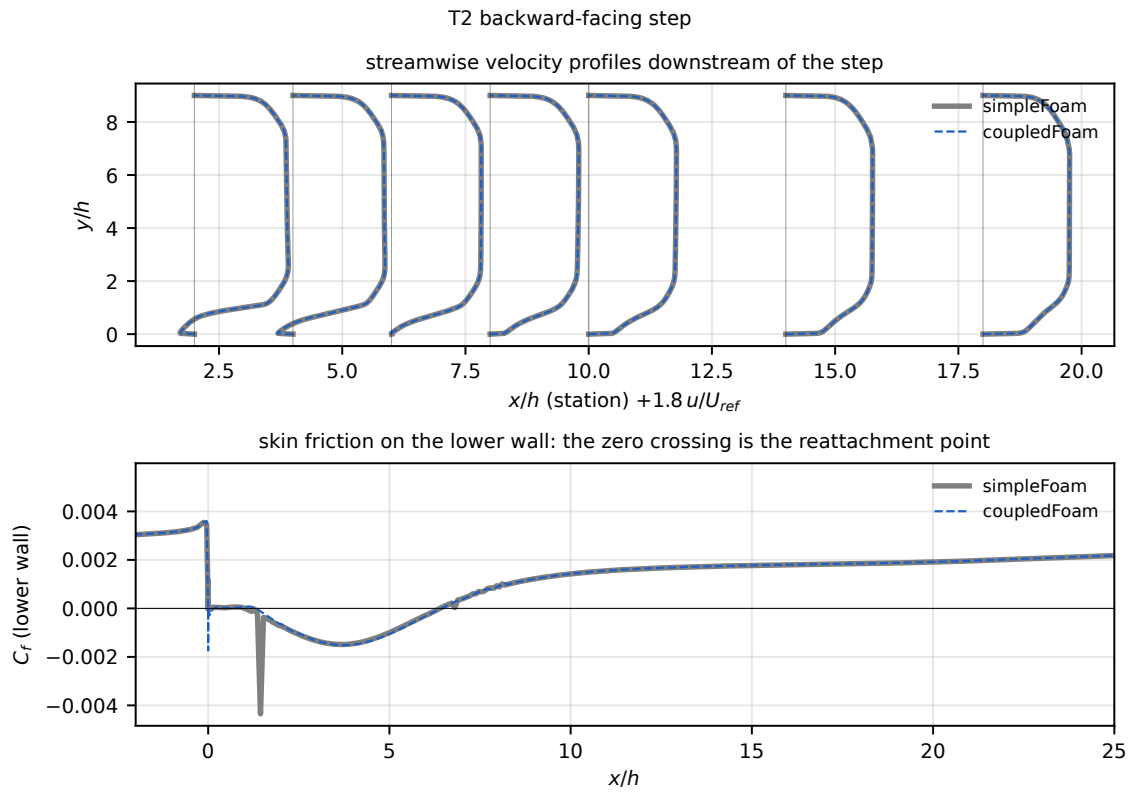


Figure 14: T2 backward-facing step: velocity profiles at the stations x/h (top) and lower-wall skin friction C_f (bottom); the zero crossing downstream of the step is the reattachment point.

T3. With GEKO the fields agree away from the airfoil (RMS 4.5e-03 in $|\mathbf{U}|/U_{ref}$), but the suction peak of coupledFoam is higher and the suction-side pressure plateau near $x/c \approx 0.1$ – 0.2 is missing (Figure 17); this is the source of the lift difference of Table 4. With SST the coupledFoam run of the test record did not converge (3000 iterations, Table 3); its field shows a large separated region with vortices above the suction side that the **simpleFoam** solution does not have (Figure 18; RMS difference 5.4e-02). The figures show the state of the test records at the time of writing, not a converged comparison.

T3 airFoil2D, GEKO: velocity magnitude

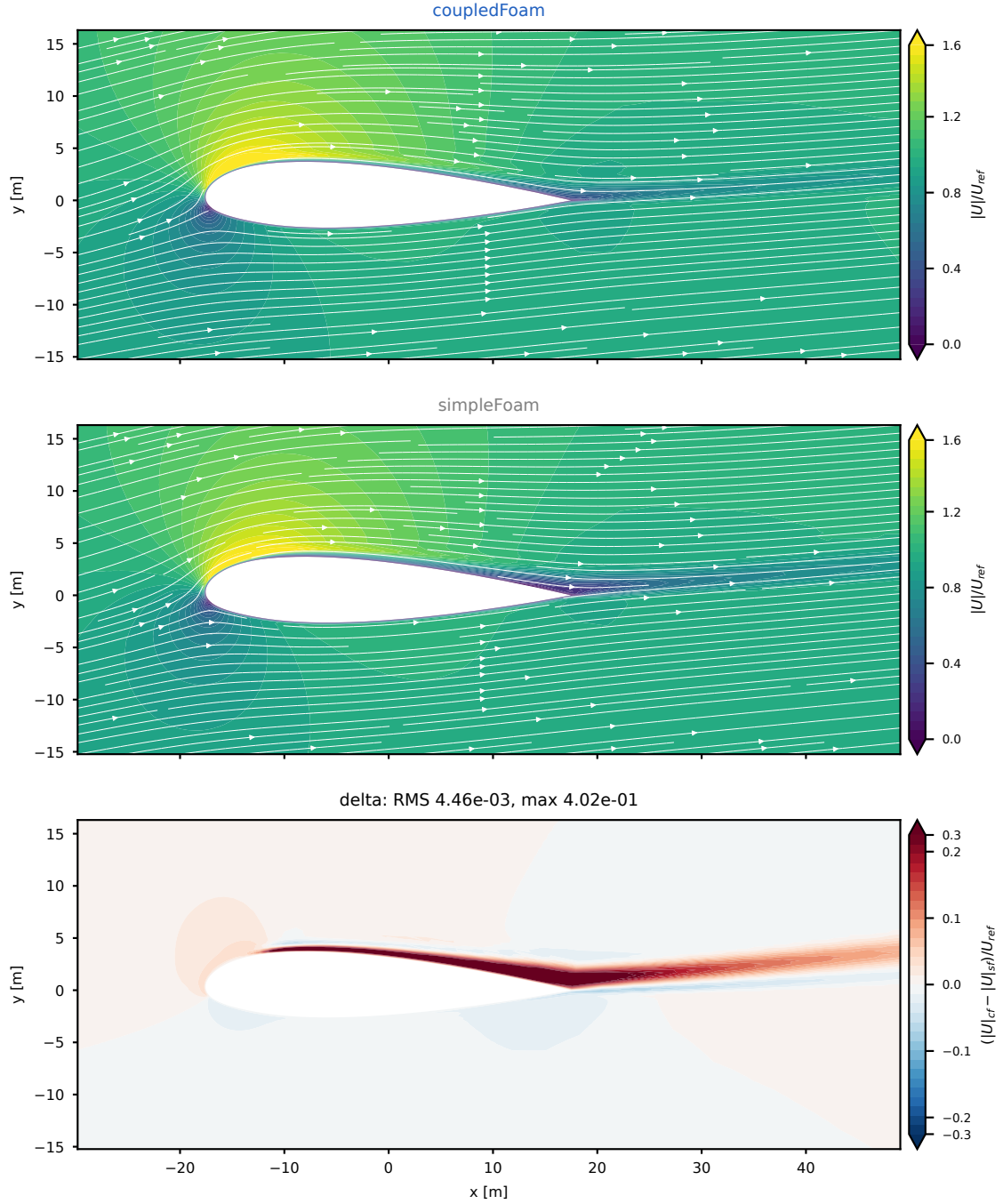


Figure 15: T3 airFoil2D, GEKO: $|\mathbf{U}|/U_{\infty}$ with streamlines around the airfoil, coupledFoam (top), simpleFoam (middle), difference (bottom).

T3 airFoil2D, GEKO: pressure coefficient

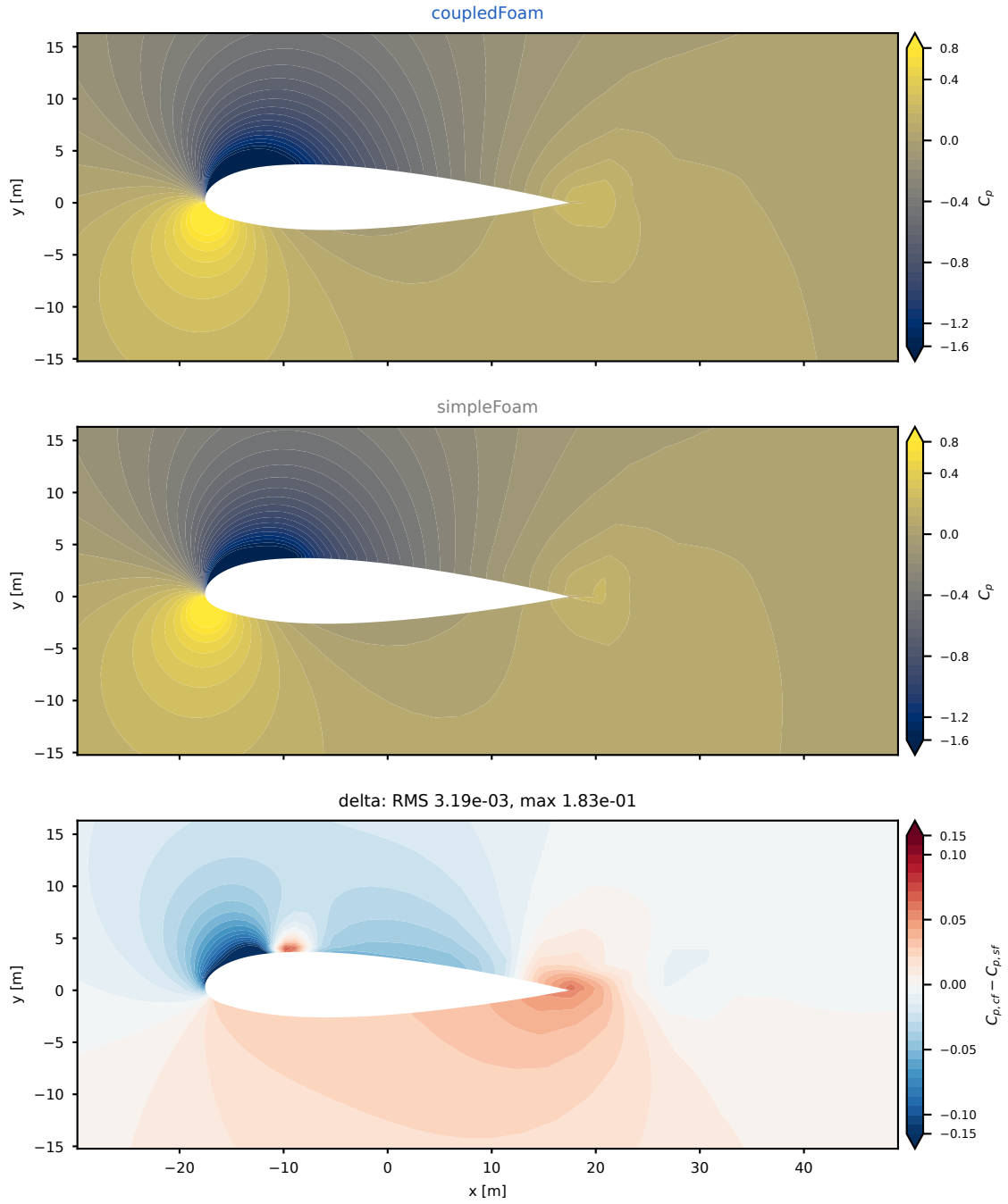


Figure 16: T3 airFoil2D, GEKO: pressure coefficient and the difference.

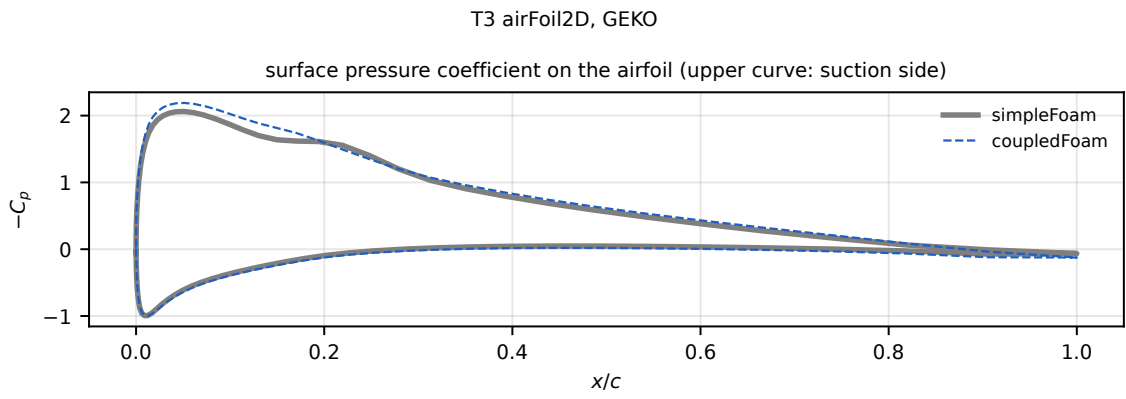


Figure 17: T3 airFoil2D, GEKO: surface pressure coefficient $-C_p$ over the chord.

T3 airFoil2D, k-omega SST: velocity magnitude

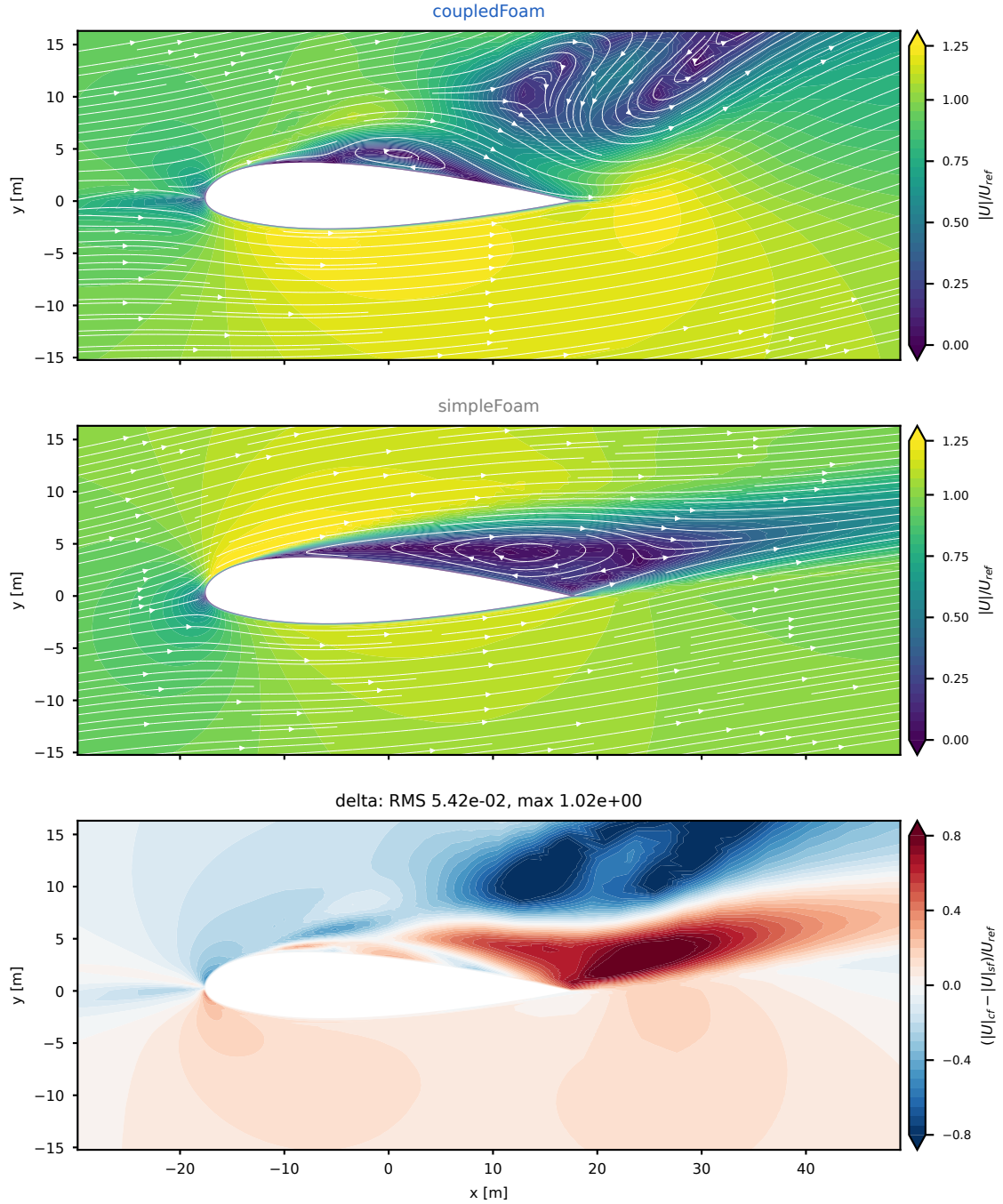


Figure 18: T3 airFoil2D, SST: $|\mathbf{U}|/U_{\infty}$ with streamlines; the coupledFoam run of this record did not converge.

T3 airFoil2D, k-omega SST: pressure coefficient

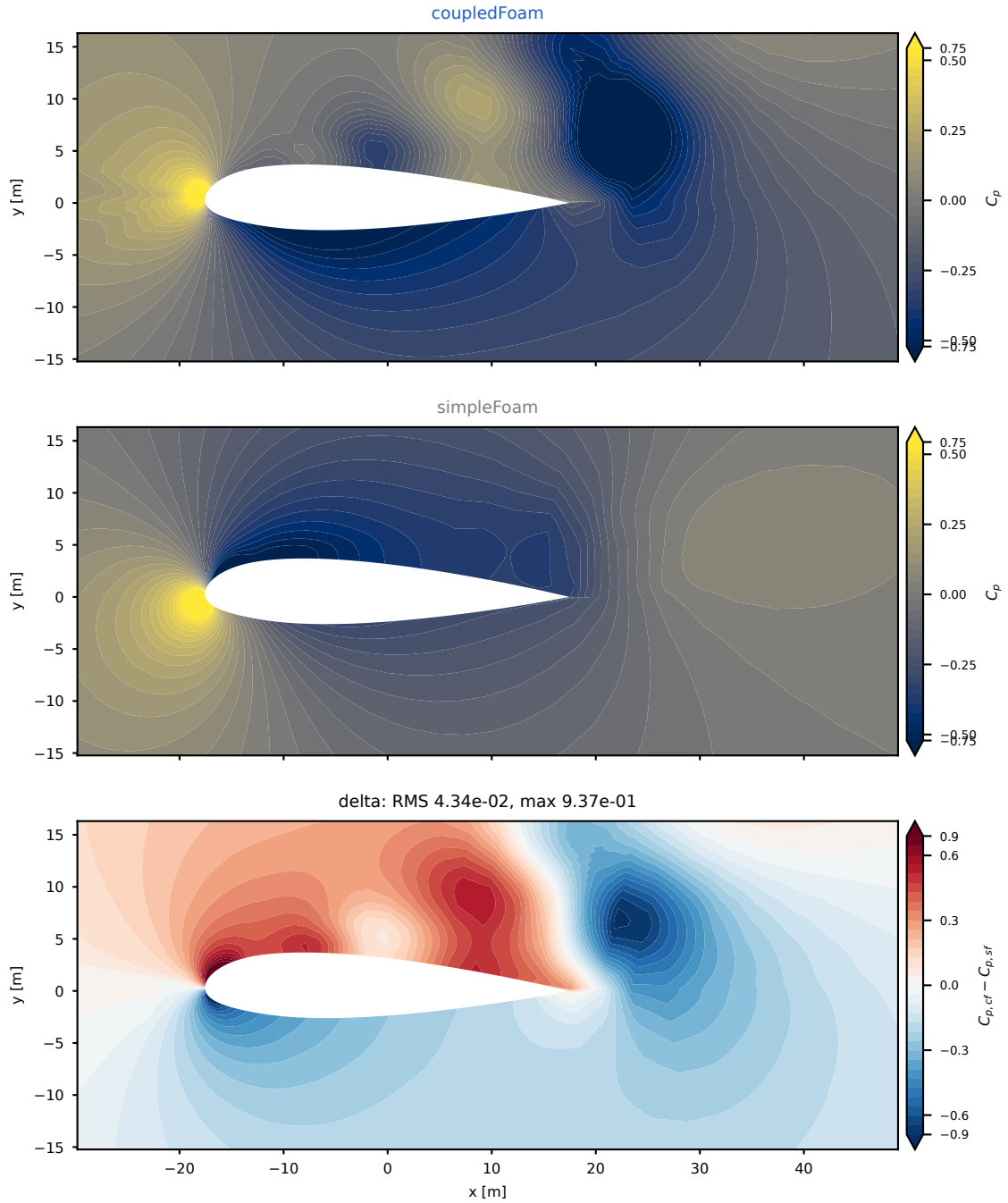


Figure 19: T3 airFoil2D, SST: pressure coefficient and the difference.

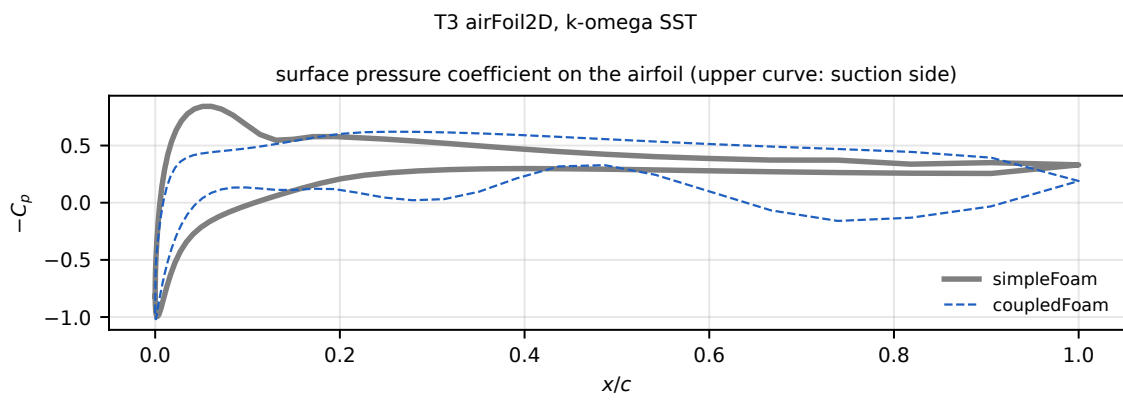


Figure 20: T3 airFoil2D, SST: surface pressure coefficient.

Results pending. Figure `figures/render_T4a_geometry` is written by `bench/make_report.py` once its data exist.

Figure 21: T4a motorBike, 354k cells: body surface mesh.

Results pending. Figure `figures/render_T4a_slice_U` is written by `bench/make_report.py` once its data exist.

Figure 22: T4a: $|\mathbf{U}|/U_\infty$ on the mid-plane (top) and the horizontal plane (bottom), `simpleFoam` (left) and `coupledFoam` (right).

T4 and T5 (3D). The 3D cases are rendered with ParaView (`bench/render_fields.py`, `pvbatch`, decomposed cases read in place): the body surface mesh, $|\mathbf{U}|/U_\infty$ and C_p on the vertical mid-plane $y = y_c$ and on a horizontal plane at 20 % of the body height, and C_p on the body, each for `simpleFoam` and `coupledFoam` with fixed colour ranges ($|\mathbf{U}|/U_\infty \in [0, 1.3]$, $C_p \in [-1.5, 1]$, identical for both solvers). Because the wake does not settle (D-042), the decisive comparison is between the window-mean fields `UMean`, `pMean` and their differences written by `coupledFieldCompare` ($|\Delta \mathbf{U}_{mean}|/U_\infty \in [0, 0.2]$, $\Delta C_{p,mean} \in [-0.2, 0.2]$). Panels whose run has not finished are drawn as “results pending”.

Results pending. Figure `figures/render_T4a_slice_p` is written by `bench/make_report.py` once its data exist.

Figure 23: T4a: C_p on the mid-plane and the horizontal plane.

Results pending. Figure `figures/render_T4a_surface_Cp` is written by `bench/make_report.py` once its data exist.

Figure 24: T4a: surface C_p on the body.

Results pending. Figure `figures/render_T4a_mean_U` is written by `bench/make_report.py` once its data exist.

Figure 25: T4a: window-mean velocity magnitude on the mid-plane, both solvers.

Results pending. Figure `figures/render_T4a_delta` is written by `bench/make_report.py` once its data exist.

Figure 26: T4a: mean-field differences: $|\Delta \mathbf{U}_{mean}|/U_\infty$ on both planes (top) and $\Delta C_{p,mean}$ on the body (bottom).

T4b motorBike (1.70M cells): body surface mesh

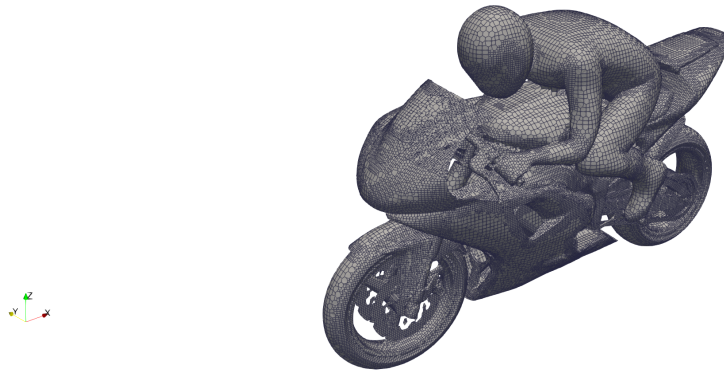


Figure 27: T4b motorBike, 1.70 M cells: body surface mesh.

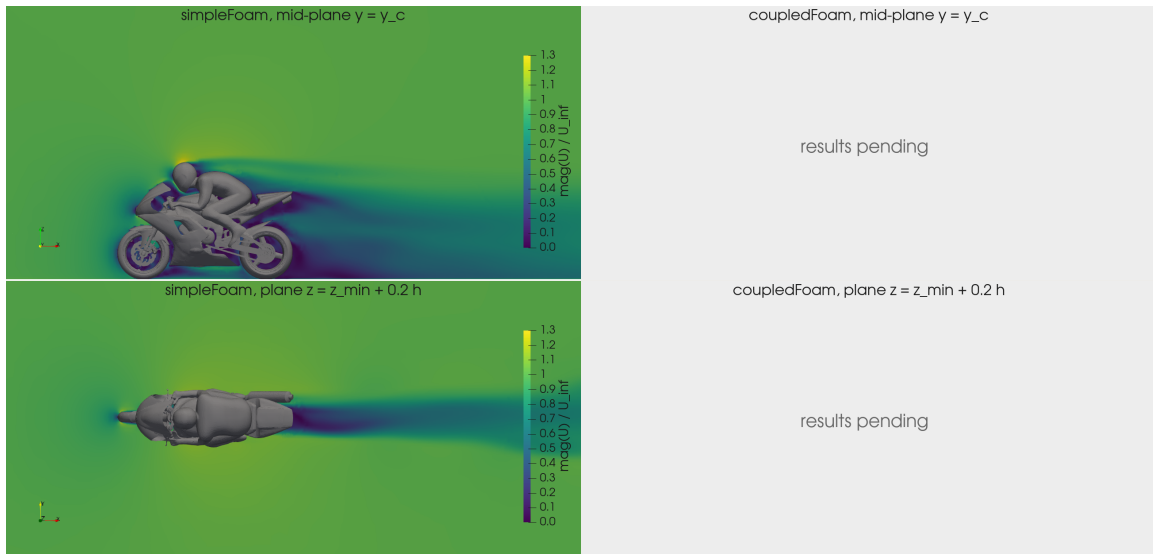


Figure 28: T4b: $|U|/U_\infty$ on the mid-plane (top) and the horizontal plane (bottom), `simpleFoam` (left) and `coupledFoam` (right).

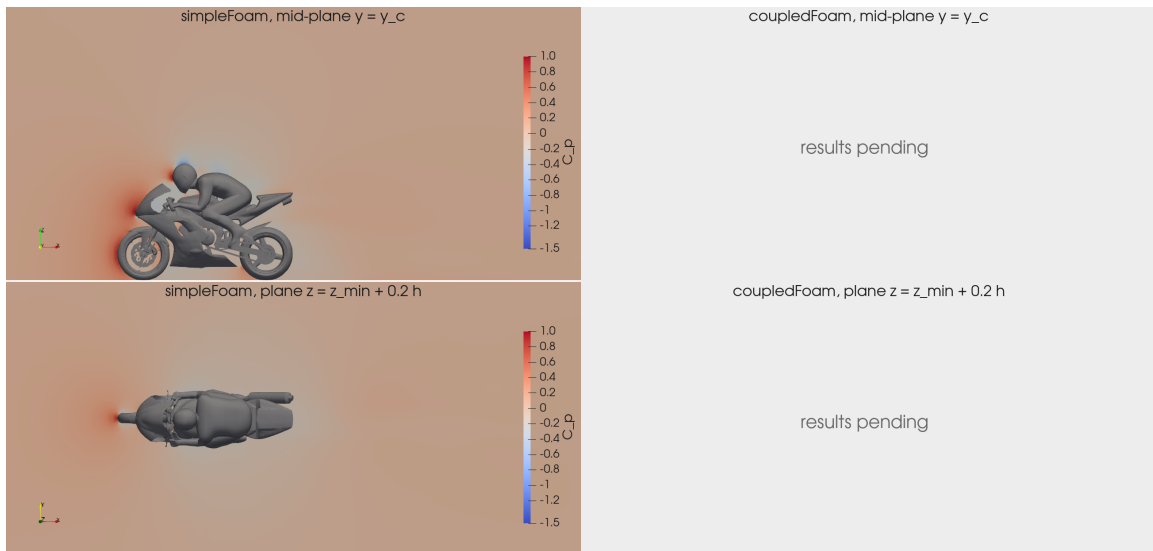


Figure 29: T4b: C_p on the mid-plane and the horizontal plane.

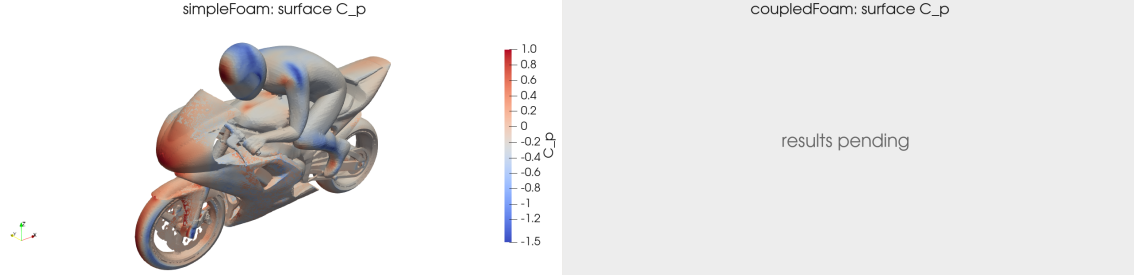


Figure 30: T4b: surface C_p on the body.

Results pending. Figure `figures/render_T4b_mean_U` is written by `bench/make_report.py` once its data exist.

Figure 31: T4b: window-mean velocity magnitude on the mid-plane, both solvers.

Results pending. Figure `figures/render_T4b_delta` is written by `bench/make_report.py` once its data exist.

Figure 32: T4b: mean-field differences: $|\Delta \mathbf{U}_{mean}|/U_\infty$ on both planes (top) and $\Delta C_{p,mean}$ on the body (bottom).

Results pending. Figure `figures/render_T5_geometry` is written by `bench/make_report.py` once its data exist.

Figure 33: T5 Ahmed body: body surface mesh.

Results pending. Figure `figures/render_T5_slice_U` is written by `bench/make_report.py` once its data exist.

Figure 34: T5: $|\mathbf{U}|/U_\infty$ on the mid-plane and the horizontal plane, both solvers.

Results pending. Figure `figures/render_T5_surface_Cp` is written by `bench/make_report.py` once its data exist.

Figure 35: T5: surface C_p on the body.

Results pending. Figure `figures/render_T5_delta` is written by `bench/make_report.py` once its data exist.

Figure 36: T5: mean-field differences on both planes and on the body.

Table 5: Benchmark configurations (D-025, amendment B10).

	solver	settings	cases
A	<code>simpleFoam</code>	tutorial settings (see text; T3: <code>SIMPLE</code> , p 0.3, $\mathbf{U}$, k , ω 0.7)	all
B	<code>simpleFoam</code>	<code>SIMPLEC</code> , relaxation p 1.0, $\mathbf{U}$ 0.9, k , ω 0.9 (tuned native baseline)	all
C	<code>coupledFoam</code>	defaults (K-cycle, controller on, Eisenstat–Walker)	all
D	<code>coupledFoam</code>	block-diagonal preconditioner (isolates the AMG)	all
E	<code>coupledFoam</code>	fixed V-cycle: <code>cycleType V</code> , <code>autoTune no</code>	T2, T4b, T5
F	<code>coupledFoam</code>	fixed relative tolerance, <code>adaptiveRelTol no</code>	T1, T3-SST
G	<code>coupledFoam</code>	Anderson acceleration, <code>anderson.enabled yes</code>	T1, T3-SST
H	<code>coupledFoam</code>	fixed K-cycle, <code>autoTune no</code> (reference for E)	T2, T4b, T5

7 Performance

7.1 Benchmark protocol

The goal is the solver that reaches a converged answer in the least time, not the one with the fewest iterations. The primary metrics are therefore the wall-clock time and the CPU time (CPU-hours summed over all ranks) to convergence, measured for all solvers with the same criterion.

Configurations. For each case in $\{T1, T2, T3\text{-SST}, T3\text{-GEKO}, T4a, T4b, T5\}$, with identical mesh, decomposition, schemes, turbulence model and boundary conditions, the configurations of Table 5 are run (D-025, amendment B10). Configuration A is the tutorial as shipped with OpenFOAM: `pitzDaily`, `backwardFacingStep2D` and `motorBike` use `SIMPLEC` (`consistent yes`) with their own relaxation factors; the `airFoil2D` tutorial uses plain `SIMPLE` with p relaxed by 0.3 and $\mathbf{U}$ and the turbulence equations by 0.7, so A on T3 overrides the case setting `consistent yes` (which serves the reference run of Section 6) with `consistent no`. The Ahmed body (T5) has no tutorial; A there means the `motorBike` tutorial settings, the closest external-aerodynamics tutorial. The native configurations start from the initial fields of the case: unlike the `motorBike` tutorial script, the benchmark does not run `potentialFoam` before `simpleFoam`. Configurations E–H isolate single components of `coupledFoam`; they run only on the cases named in B10 (E and H on T2, T4b, T5; F and G on T1 and T3-SST). E fixes the V-cycle by also switching the cycle-efficiency controller off, since the controller would otherwise promote V to F and W (Section 4.3); because C differs from E in the cycle *and* in the controller, configuration H (K-cycle, controller off) is added as the clean reference for the cycle comparison. The harness records the final cycle type and post-smoothing count of every `coupledFoam` run.

Stop criterion. The solvers’ own stop criteria are disabled and a fixed iteration budget is run. The harness determines the convergence iteration identically for all solvers: over the last 100 iterations, $\max C_d - \min C_d \leq 0.002 |C_d|$ and the same for C_l (the pressure drop for T1 and T2).

Wake cases (D-042 and addendum). T4a, T4b and T5 have physically oscillating wakes: the T4b `simpleFoam` reference oscillates by about $\pm 1\%$ in C_d and $\pm 8\%$ in C_l indefinitely, so

the window criterion above cannot be met by any steady solver on these cases. For them (user decisions, D-042 and its addendum) a run of n iterations is converged when its window mean is stationary: over the last $W = \max(1000, n/2)$ iterations ($W \leq n$) the means of the two half-windows differ by at most $\max(0.01 |\bar{C}|, 0.005)$, for C_d and C_l separately. The first version of the rule ($W = \max(500, n/4)$, $\max(0.005 |\bar{C}|, 0.002)$) was too tight. Over $W = 1000$ the half-window means of the 4000-iteration T4b reference still drift by 5.5 % in C_l ; over $W = 2000$ they drift by 0.9 % (C_d) and 1.1 % (C_l). The convergence iteration is the first iteration (scanned in steps of 50) at which this holds, and the coefficients compared are the window means, within $\max(2\%, 0.002)$ for C_d and $\max(2\%, 0.01)$ for C_l . Numbers alone can agree for the wrong reason, so the flow fields are compared as well. A `fieldAverage` function object averages $\mathbf{U}$ and p over the same window W . The utility `coupledFieldCompare` writes the differences of the mean fields between the two solvers, including the boundary values (the surface ΔC_p on the body), and reports their volume-weighted RMS. The proposed pass thresholds are an RMS of $|\Delta \bar{\mathbf{U}}|/U_\infty$ and of $|\Delta \bar{p}|/(\frac{1}{2}U_\infty^2)$ of at most 0.02 each. The cached references, which were run before the averaging existed, are continued once by W iterations with the averaging active. The criteria of T0–T3 are unchanged.

Timing. Every rank of every application that the case script starts runs under `/usr/bin/time -v`. The total cost of a solution is the wall time (maximum over ranks) and the CPU time (user plus system, summed over ranks, in CPU-hours) of the solver *plus* that of its pre-processing: for `coupledFoam` this is the `potentialFoam` initialisation, which the case script runs for `coupledFoam` only. The pre-processing runs before the solver, so its wall and CPU times are added; both parts are also recorded separately in every result. The time to convergence is

$$t_{\text{conv}} = t_{\text{pre}} + f t_{\text{solver}}, \quad c_{\text{conv}} = c_{\text{pre}} + f c_{\text{solver}}, \quad (25)$$

with t the wall and c the CPU time, and f the solver’s own wall-clock progress at the convergence iteration (`coupledFoam`’s cumulative wall time per iteration, `simpleFoam`’s `ClockTime`) divided by its total (D-025). The pre-processing is a fixed offset and is not scaled. Using the wall-clock fraction f also for the CPU time assumes a constant CPU-to-wall ratio during the run. The time per iteration (Figure 40) is that of the solver alone. Peak memory is the per-rank maximum RSS, reported as sum and maximum over ranks. Each configuration is run three times and the median (with minimum and maximum) is reported. Speed-up is reported both as a wall-time and as a CPU-hour ratio against the better of A and B. Results are tagged with a hash of their configuration; results of an outdated configuration are excluded from all tables and figures.

Acceptance of the B10 variants. For F, the adaptive tolerance of C must not be more than 5 % slower than the fixed tolerance, in wall time and in CPU-hours, and the monitored quantity (C_d , or Δp for T1) at the end of the fixed iteration budget must agree to 10^{-4} (relative). E, G and H have no threshold; their differences to C are reported in wall time and CPU-hours.

Hardware. 16 physical cores (32 logical with SMT), 94 GB RAM visible to WSL, of which at most 10 cores are used (D-031); MPI ranks are bound to physical cores (`-bind-to core -map-by core`); benchmarks run only when no other solver runs, and the load average before each run is recorded (D-007).

7.2 Results

The executive summary (Table 6) compares the best native configuration with `coupledFoam` in wall-clock time and CPU-hours; the per-configuration results are in Figures 37 (wall time) and 38 (CPU-hours), the iteration counts in Figure 39. Figure 40 splits the time per `coupledFoam` iteration, and Figure 41 compares the measured peak memory with the budget of Table 1.

Table 6: Executive summary: time to convergence (identical window criterion, coupledFoam including its potentialFoam initialisation), best native configuration against coupledFoam, as wall-clock time and CPU-hours.

Results pending. Table `tables/executive_summary.tex` is written by `bench/make_report.py` once its data exist.

Results pending. Figure `figures/bench_wall` is written by `bench/make_report.py` once its data exist.

Figure 37: Wall-clock time to convergence (median of three runs) per case and configuration.

The acceptance of the B10 variants is evaluated in Table 7, with the cycle comparison (E and H against C) in Figure 42 and Anderson acceleration (G) in Figure 43. The block-GAMG hierarchies are listed in Table 8, the Eisenstat–Walker forcing terms and preconditioner efficiencies are shown in Figure 46, and the strong scaling in Figure 44.

Results pending. Figure `figures/bench_cpu` is written by `bench/make_report.py` once its data exist.

Figure 38: CPU time to convergence (CPU-hours summed over ranks, median of three runs).

Results pending. Figure `figures/bench_iters` is written by `bench/make_report.py` once its data exist.

Figure 39: Outer iterations to convergence. Shown for completeness; the decisive metrics are those of Figures 37 and 38.

Results pending. Figure `figures/bench_breakdown` is written by `bench/make_report.py` once its data exist.

Figure 40: coupledFoam time per outer iteration split into assembly, linear solve and turbulence: wall time (left) and CPU time summed over ranks (right, split in the proportions of the wall-time breakdown).

Results pending. Figure `figures/bench_memory` is written by `bench/make_report.py` once its data exist.

Figure 41: Peak memory, sum of the per-rank maximum RSS, with the B7 budget of Table 1 (105–120 GB at 45 M cells) scaled to the cell count of each case (grey band). On small meshes the fixed per-process overhead of OpenFOAM dominates, so the comparison is meaningful only for the larger cases.

Table 7: Amendment B10: configurations E, F, G and H against C in wall time and CPU-hours; pass/fail for F.

Results pending. Table `tables/b10_acceptance.tex` is written by `bench/make_report.py` once its data exist.

Results pending. Figure `figures/bench_cycles` is written by `bench/make_report.py` once its data exist.

Figure 42: Benchmark cycle comparison on T2, T4b and T5: default K-cycle with controller (C), fixed K-cycle (H) and fixed V-cycle (E), wall time and CPU-hours to convergence.

Results pending. Figure `figures/anderson` is written by `bench/make_report.py` once its data exist.

Figure 43: Anderson acceleration on and off (T1, T3-SST): outer iterations, wall time and CPU-hours to convergence.

Results pending. Figure `figures/scaling` is written by `bench/make_report.py` once its data exist.

Figure 44: Strong scaling on T4b with 1, 2, 4, 8 and 10 ranks (D-031): wall time per iteration, parallel efficiency and CPU-hours of the fixed iteration count, coupledFoam and simpleFoam. Pass: the parallel efficiency of coupledFoam at the largest rank count is at least 80 % of that of simpleFoam.

Table 8: Block-GAMG hierarchy per run: cycle type and post-smoothing sweeps at start and end (autoTune), levels, operator complexity, cells, ranks and coarsening ratios per level, coarsest-level solver. Ranks per level are n/a for logs written before D-030.

run	cycle (start/end)	post-sweeps	levels	mergeLevels	C_op	cells per level	ranks per level	coarsening ratios	coarsest solver
T0_Re1000_np1	K/K	2/2	4	2	1.325	16384 4096 1024 256	1 1 1 1	4 4 4	blockBiCGStab
T0_Re1000_np4	K/K	2/2	3	2	1.310	16384 4096 1024	4 1 1	4 4	blockBiCGStab
T0_Re100_np1	K/K	2/2	4	2	1.325	16384 4096 1024 256	1 1 1 1	4 4 4	blockBiCGStab
T0_Re100_np4	K/K	2/2	3	2	1.361	16384 4082 1008	4 1 1	4.01 4.05	blockBiCGStab
T1_np1	V/V	2/2	4	2	1.412	12225 3052 754 373	1 1 1 1	4.03 4.07 2.03	blockBiCGStab
T1_np4	V/V	2/2	3	2	1.446	12225 3049 1518	4 1 1	4.03 2.02	blockBiCGStab
T2_np1	V/V	2/2	4	2	1.401	20540 5128 1267 299	1 1 1 1	4.02 4.07 4.2	blockBiCGStab
T2_np4	V/V	2/2	3	2	1.371	20540 5117 1245	4 1 1	4.02 4.1	blockBiCGStab
T3_GEKO_np1	K/K	2/4	3	3	1.205	10720 1330 328	1 1 1	8.06 4.05	blockBiCGStab
T3_kOmegaSST_np1	K/K	2/4	3	3	1.205	10720 1330 328	1 1 1	8.06 4.05	blockBiCGStab

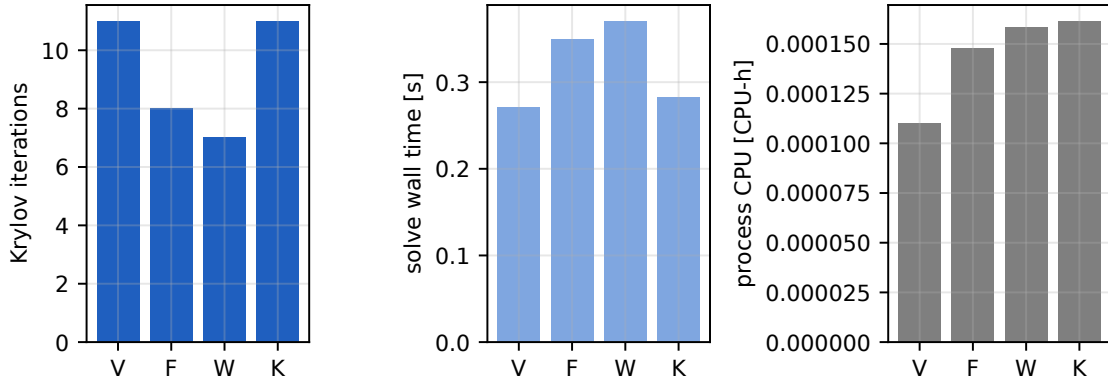


Figure 45: Unit test **Test-blockGAMG** (not a benchmark): Krylov iterations, solve wall time and process CPU time of the V-, F-, W- and K-cycles on the block-Poisson system of the 128×128 cavity mesh, serial, solved to 10^{-9} (D-023).

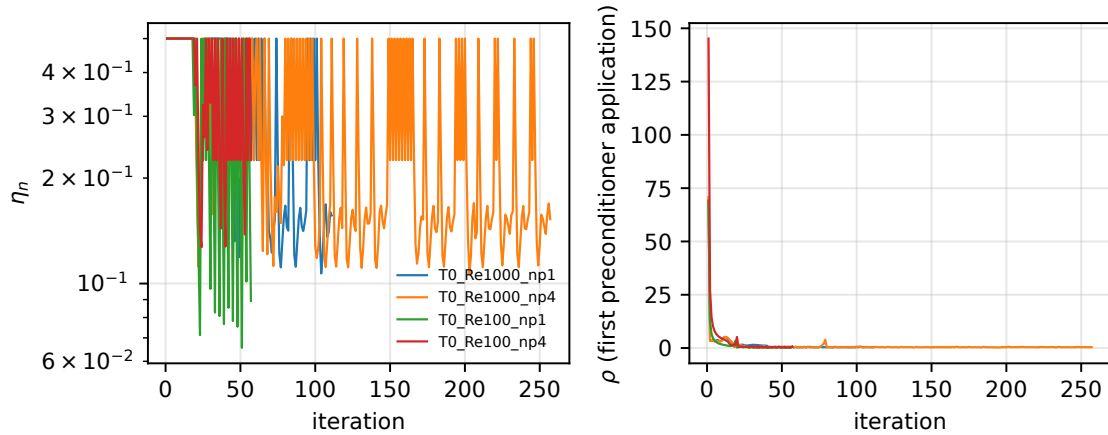


Figure 46: Eisenstat-Walker forcing term η_n and preconditioner efficiency ρ with the controller events.

7.3 Speed-up measured on the test runs

Until the benchmark matrix has run, the test records give a first measurement of the speed-up (single serial runs on the shared machine, not benchmark quality; D-007). Each solver is stopped by its own test criterion: `coupledFoam` at the residual target of the test ($R < 10^{-8}$ for T0, 10^{-5} for T1 and T2), `simpleFoam` at its `residualControl`. The residual normalisations differ (Section 3.2), so the time axis, not the residual level, is the comparable quantity; the field comparisons of Section 6.3 show that both runs end at the same solution. Serial runs make CPU-hours and wall-clock time nearly proportional; both are reported.

Figure 47 plots the residuals against wall-clock time. Figure 48 gives the time and CPU-hours to convergence: `coupledFoam` is faster by a factor of 30.1 in wall-clock time (28.1 in CPU-hours) on T0 at $Re = 100$, 6.9 (7.0) at $Re = 1000$ and 3.0 (3.0) on T1 (21.23s against 64s). On T2 `simpleFoam` did not reach the target within 20000 iterations, so the factor 11.3 (11.3 in CPU-hours) is a lower bound. On T3 neither solver reached its residual criterion within its iteration limit; there the bars compare the run lengths only. Figure 49 shows why the iteration count alone is misleading: one coupled iteration costs 3.3 times a `simpleFoam` iteration on T2 and 9.1 times on T3-GEKO, and the linear solve takes 74 % (T2) to 89 % (T3-GEKO) of the iteration. Figure 50 places both effects in one picture.

8 Discussion and limitations

8.1 What single precision requires

Four observations made during development determine whether a float block solver reaches double-precision steady residuals. (i) The residual must be exact: $b - Ax$ is formed in double from the double assembly (D-012). With that, the float matrix only affects the convergence rate, as in mixed-precision iterative refinement. (ii) Near-singular systems must be made nonsingular exactly. A weak pressure reference leaves a near-null space in which the float solve drifts once the right-hand side is rounded; the strong reference of D-021 removes it without approximation. (iii) Linear solves near the float floor can fail. A best-iterate return (D-020) and treating a solve that misses its target as a failed step with a CFL cut (D-029) keep a converged state from being damaged. (iv) Tolerances must be consistent with the float floor and with the outer target. The absolute linear tolerance must lie below the outer target, and at least one Krylov iteration must always be made (D-022). Conversely, the float floor limits how closely two solves can agree: two solves of the block Poisson test stopped at $\|\mathbf{r}\|_2/\nu_n = 10^{-8}$ differ by 1.3×10^{-5} (relative L_2) between 1 and 4 ranks, 9.3×10^{-7} at 10^{-9} , and tighter tolerances do not reduce the difference below about 6×10^{-7} (D-023; values from the table of that entry, the logs were not kept). Cross-rank identity of float solves must therefore be tested at a tolerance matched to that floor.

The precision types are the library's own, because OpenFOAM's SPDP build is the inverse of what a memory-bound block solver needs (D-001).

8.2 Deferred correction at high CFL

With frozen flux, exact linear solves and $\omega = 1$, one outer iteration with deferred correction is the stationary iteration $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{M}^{-1}(\mathbf{b} - \mathbf{A}_{\text{HO}}\mathbf{x}_k)$ with $\mathbf{M} = \mathbf{A}_{\text{UD}} + \mathbf{T}$, $\mathbf{T} = \text{diag}(V/\Delta t)$. Its error propagates with

$$\mathbf{G} = \mathbf{I} - (\mathbf{A}_{\text{UD}} + \mathbf{T})^{-1}\mathbf{A}_{\text{HO}} = (\mathbf{A}_{\text{UD}} + \mathbf{T})^{-1}(\mathbf{T} - \mathbf{C}), \quad \mathbf{C} = \mathbf{A}_{\text{HO}} - \mathbf{A}_{\text{UD}}, \quad (26)$$

and converges only if $\rho(\mathbf{G}) < 1$. The pseudo-time term plays two roles: it damps $\mathbf{C}$ relative to $\mathbf{T}$, and it slows the smooth modes. As $\text{CFL} \rightarrow \infty$, $\mathbf{T} \rightarrow 0$ and $\mathbf{G} \rightarrow -\mathbf{A}_{\text{UD}}^{-1}\mathbf{C}$, so the damping by the pseudo-time term disappears exactly when the CFL control removes it.

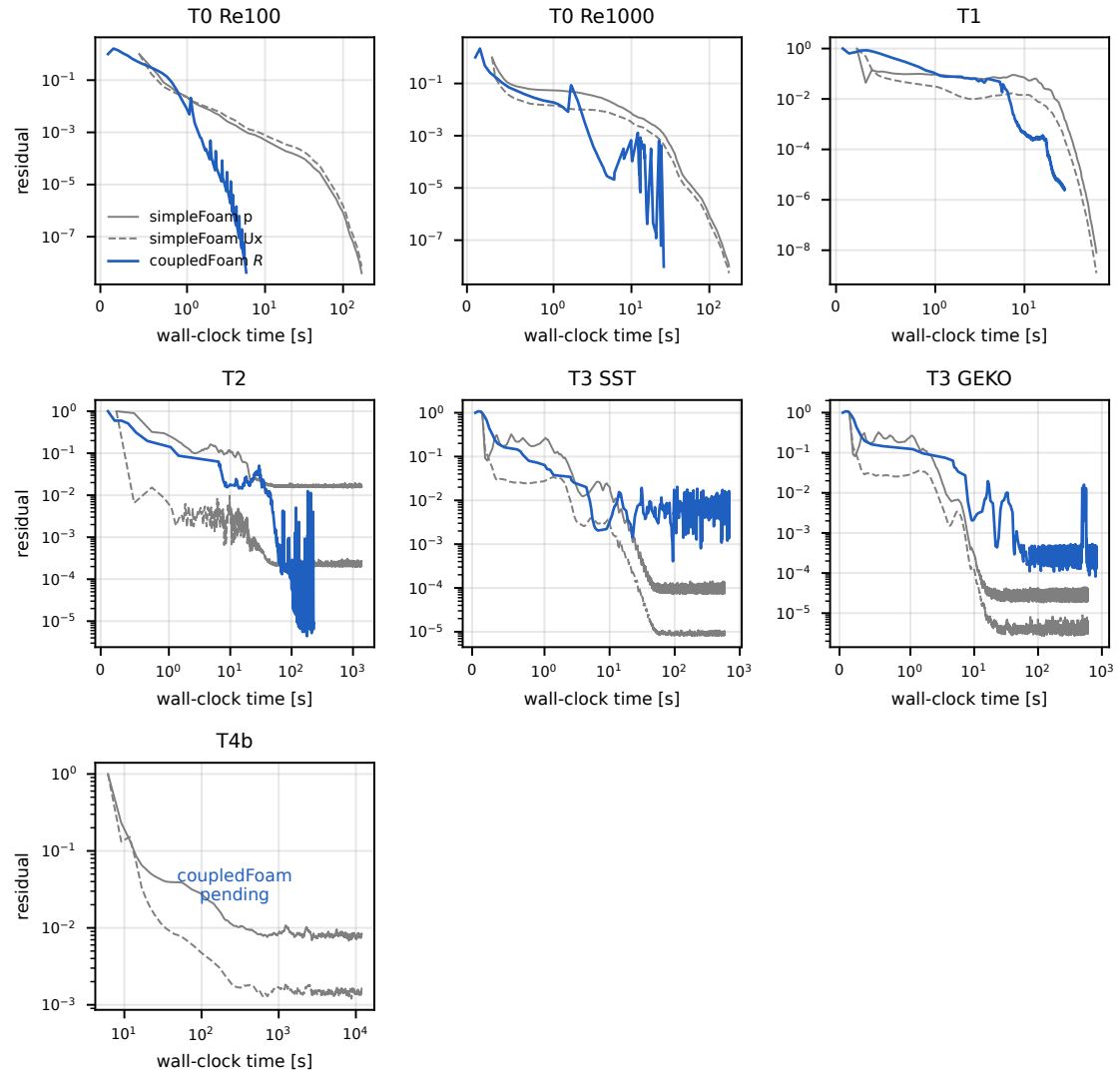


Figure 47: Residual against wall-clock time (symmetric log time axis): coupledFoam combined residual R and simpleFoam initial residuals of p and U_x . Normalisations differ; the time axis is the comparable quantity. Cases whose coupledFoam run is not finished show the reference only.

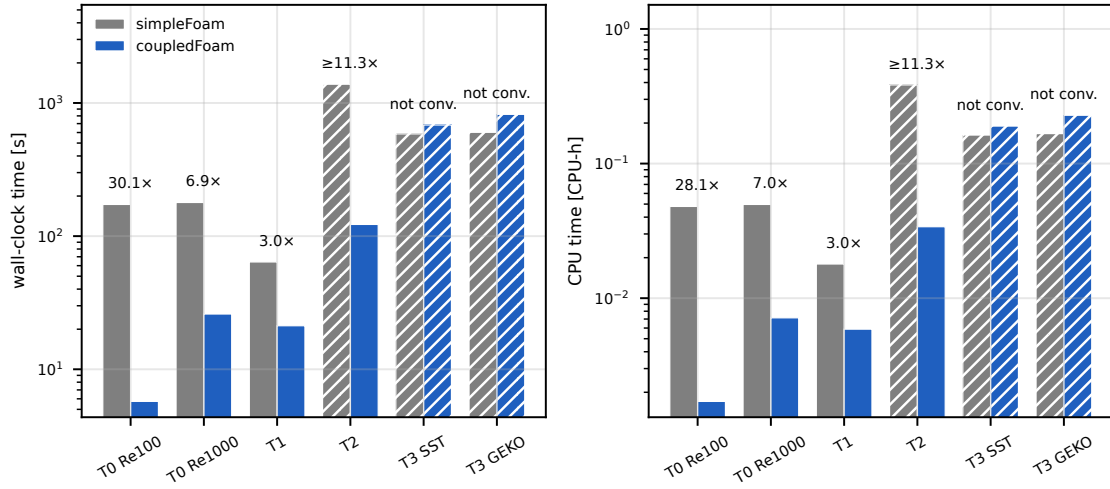


Figure 48: Wall-clock time (left) and CPU-hours (right) to convergence, simpleFoam and coupledFoam, test records, serial. Numbers: speed-up simpleFoam/coupledFoam. Hatched bars: the criterion was not reached, the bar is the whole run; $\geq$: lower bound.

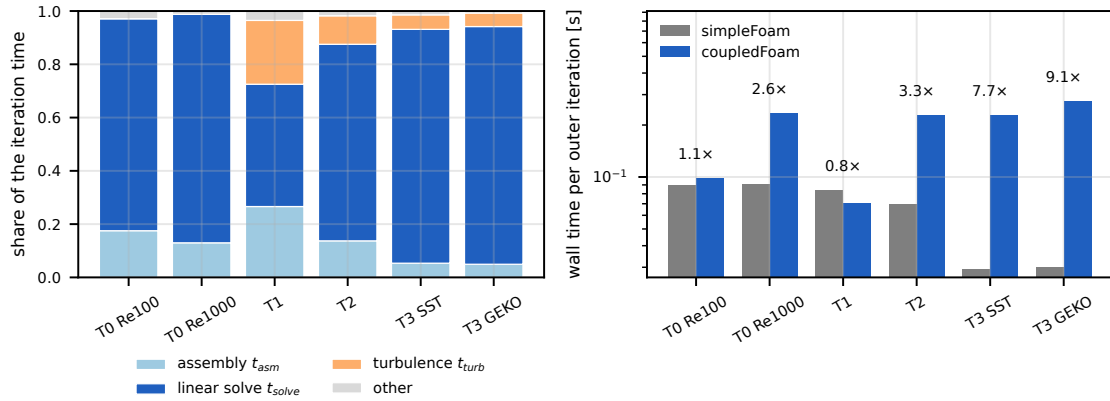


Figure 49: Time per outer iteration. Left: share of assembly, linear solve and turbulence in a coupledFoam iteration. Right: wall time per iteration of both solvers and the cost ratio coupledFoam/simpleFoam.

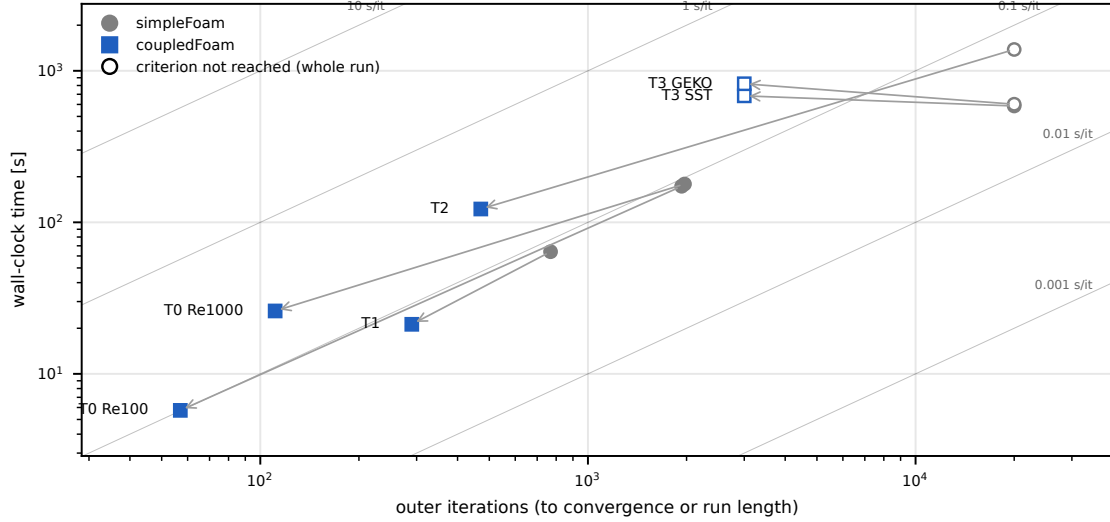


Figure 50: Iterations against wall-clock time. Each arrow leads from simpleFoam to coupledFoam for one case; diagonals are lines of constant time per iteration. coupledFoam trades far fewer iterations for more expensive ones; it is faster where the arrow points down.

For a first estimate we evaluated the symbol of (26) for constant convection at angle θ on a uniform Cartesian grid, with OpenFOAM’s `linearUpwind` (face value from the upwind cell plus its central gradient, which in 1D is Fromm’s scheme), first-order upwind in A_{UD} , central diffusion at the cell Péclet number Pe_h , and T from (13) with $V^{1/3} = h$; modes with wavelength longer than the domain ($|\mathbf{k}| < 2\pi/(128h)$) are excluded, all other wave vectors in $[-\pi/h, \pi/h]^2$ are sampled. The script `bench/exploratory_numbers.py` computes the values quoted here. For 1D convection the symbol of $A_{UD}^{-1}C$ is $\frac{1}{2}\sin\xi$, so $\rho \leq \frac{1}{2}$ even at infinite CFL. In 2D, cross-stream modes whose upwind operator consists almost entirely of numerical diffusion, which the correction removes, bring ρ close to one. In the limit $CFL \rightarrow \infty$ at $\theta = 45^\circ$ we find $\rho(G) = 0.73$ for $Pe_h = 7.8$ (the $Re = 1000$ cavity on 128^2 cells, based on the lid velocity) and $\rho(G) = 0.96$ for $Pe_h = 78$. At CFL 500 and $Pe_h = 7.8$ the value rises to 0.92: at finite CFL the largest $|G|$ comes from the smoothest resolvable modes, which the pseudo-time term slows, not from the correction. In this idealised model, therefore, $\rho < 1$ at every CFL, but with a small margin at high cell Péclet numbers. Effects outside the model can push the iteration over the edge: non-uniform and distorted cells, boundaries and the singular corners of the cavity, the Picard lag of the convecting flux, and above all an inexact linear solve, which replaces M^{-1} by a different approximate inverse.

This is consistent with what we observed on the $Re = 1000$ cavity in exploratory runs (logs in `run/exp_re1000`, outside the test protocol). With the solver default of that time (BiCGStab preconditioned by block GAMG with a Gauss–Seidel smoother, relative tolerance 0.05), the run stagnated once deferred correction was switched on after the upwind start-up: after the start-up the CFL number stayed between 3.5 and 40 (5th to 95th percentile 3.9 to 24) with 29 CFL cuts, and after 300 iterations R was 3.4×10^{-4} (minimum 2.2×10^{-6}). With pure upwind throughout, the same configuration reached CFL 500 at iteration 154 (14 CFL cuts in total) and $R < 10^{-8}$ in 186 iterations. With the block ILU(0) smoother and otherwise identical settings, deferred correction reached CFL 500 at iteration 18 with 0 cuts and converged to $R < 10^{-8}$ in 59 iterations. The stagnation is thus not intrinsic to deferred correction at CFL 500 for this case; it arises from a small theoretical margin combined with an inaccurate inner solve. The design consequences are the Eisenstat–Walker forcing (tighter solves as the outer iteration converges), the stronger smoothers and cycles of Section 4.3, the treatment of failed solves, and remediation,

which sets $\beta_P = 0$ where the correction is locally destabilising. A rigorous statement for the actual discretisation would require the spectrum of (26) for the assembled operators, which we have not computed.

On the $Re = 100$ cavity an exploratory comparison of linear solvers (logs in `run/exp_linsolver`) showed the practical weight of the inner solve: BiCGStab with the Gauss–Seidel smoother needed 59 outer iterations and 2319 Krylov iterations in total, with 8 solves reaching the iteration limit, whereas the ILU(0) smoother needed 45 outer and 389 Krylov iterations and reduced the summed solve time by a factor of 5.2 (134 s against 26 s); GMRES with the Gauss–Seidel smoother was similarly fast (25 s, 50 outer iterations). These runs were made on a loaded machine and are indicative only; the benchmark of Section 7 is authoritative.

8.3 Other limitations

- *Pseudo-time in the Rhie–Chow coefficient.* $D = V/\bar{a}$ includes the pseudo-time term, so the converged solution depends weakly on the final local time step through the Rhie–Chow dissipation, as SIMPLE solutions depend on the relaxation factor. At the large CFL numbers reached at convergence the effect is small; the cavity comparison (Section 6) bounds it for that case.
- *Explicit cyclic and AMI coupling.* Cyclic, cyclicAMI and processor-cyclic interfaces are lagged. Periodic cases converge more slowly; implicit block coupling across them is Phase 2.
- *Per-cell gradient limiting* applies to the Rhie–Chow pressure gradient only; the gradient inside the user’s convection scheme cannot be limited per cell (D-018).
- *Boundary conditions.* Outflow through a fixed-value velocity face uses the native boundary value (D-019); `inletOutlet` and `totalPressure` are linearised about the previous outer iteration.
- *MRF* is implicit but gated by an informational test only (D-009).
- *Atomic write* is implemented at file level (the restart dictionary is renamed last) and skipped with the collated file handler (D-017).
- *Scope and hardware.* The 45 M-cell production case and its memory budget cannot be verified on the development machine (94 GB, D-007); the largest meshes of the test suite are T4b and T5.
- *Residual definitions* of `coupledFoam` and `simpleFoam` differ; the iteration criterion of T2 is defined in D-024, and all timing comparisons use the solver-independent force window (for the wake cases T4 and T5 the stationary window mean, D-042).

8.4 Phase 2

Block-selective AMG [33, 34] as an additional preconditioner; adaptive mesh refinement with rebalancing (the restart format already reserves the refinement history); a single-precision PETSc field-split reference; implicit cyclic and AMI block coupling; GPU execution; and the 45 M-cell production run with its memory budget verified by peak RSS.

9 Conclusions

`coupledFoam` solves the steady incompressible RANS equations in OpenFOAM with a fully coupled 4×4 -block pressure–velocity system, implicit boundary conditions through the native patch linearisation, implicit MRF, and a continuity row that is by construction the mass balance of the

output flux. Storing the block system and all Krylov and multigrid vectors in single precision while forming the residual and all reductions in double turns the outer iteration into a mixed-precision defect correction: on the cavity test the solver converges to $R < 10^{-8}$ and reproduces the native solution to 2.9×10^{-6} . Robustness comes from pseudo-transient continuation with local, solution-limited time steps, a physicality line search, cell-set remediation, rollback and the explicit handling of failed linear solves; efficiency is sought from a block additive-correction AMG with K-cycles under flexible GMRES and Eisenstat–Walker forcing. The exploratory cavity runs show that the accuracy of the inner solve decides whether deferred correction survives at high CFL numbers, which is why the linear solver and the globalisation are designed together. Whether the coupled solver is faster than tuned native SIMPLEC in wall-clock time and CPU-hours is answered by the benchmark of Section 7, whose results are generated directly from the measurement records.

Reproducibility. All measured numbers, tables and figures are generated by `bench/make_report.py` from the JSON records in `results/`; the numbers of the exploratory runs and of the symbol analysis in Section 8 are extracted or computed by `bench/exploratory_numbers.py` (with the source of every value recorded in `results/exploratory/`). The title page states the commit (dca2666) at which they were generated.

References

- [1] S. R. Ahmed, G. Ramm, and G. Faltin. Some salient features of the time-averaged ground vehicle wake. Technical Report 840300, SAE, 1984.
- [2] D. G. Anderson. Iterative procedures for nonlinear integral equations. *Journal of the ACM*, 12(4):547–560, 1965.
- [3] M. Baboulin, A. Buttari, J. Dongarra, J. Kurzak, J. Langou, J. Langou, P. Luszczek, and S. Tomov. Accelerating scientific computations with mixed precision algorithms. *Computer Physics Communications*, 180(12):2526–2533, 2009.
- [4] E. Carson and N. J. Higham. Accelerating the solution of linear systems by iterative refinement in three precisions. *SIAM Journal on Scientific Computing*, 40(2):A817–A847, 2018.
- [5] M. Ceze and K. J. Fidkowski. A robust adaptive solution strategy for high-order implicit CFD solvers. In *20th AIAA Computational Fluid Dynamics Conference*, 2011. AIAA Paper 2011-3696.
- [6] M. Ceze and K. J. Fidkowski. Pseudo-transient continuation, solution update methods, and CFL strategies for DG discretizations of the RANS-SA equations. In *21st AIAA Computational Fluid Dynamics Conference*, 2013. AIAA Paper 2013-2686.
- [7] M. Ceze and K. J. Fidkowski. Constrained pseudo-transient continuation. *International Journal for Numerical Methods in Engineering*, 102(11):1683–1703, 2015.
- [8] M. Darwish, I. Sraj, and F. Moukalled. A coupled incompressible flow solver on structured grids. *Numerical Heat Transfer, Part B: Fundamentals*, 52(4):353–371, 2007.
- [9] M. Darwish, I. Sraj, and F. Moukalled. A coupled finite volume solver for the solution of incompressible flows on unstructured grids. *Journal of Computational Physics*, 228(1):180–201, 2009.
- [10] D. M. Driver and H. L. Seegmiller. Features of a reattaching turbulent shear layer in divergent channel flow. *AIAA Journal*, 23(2):163–171, 1985.

- [11] S. C. Eisenstat and H. F. Walker. Choosing the forcing terms in an inexact Newton method. *SIAM Journal on Scientific Computing*, 17(1):16–32, 1996.
- [12] J. H. Ferziger and M. Perić. *Computational Methods for Fluid Dynamics*. Springer, Berlin, 3rd edition, 2002.
- [13] U. Ghia, K. N. Ghia, and C. T. Shin. High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method. *Journal of Computational Physics*, 48(3):387–411, 1982.
- [14] B. R. Hutchinson and G. D. Raithby. A multigrid method based on the additive correction strategy. *Numerical Heat Transfer*, 9(5):511–537, 1986.
- [15] C. T. Kelley and D. E. Keyes. Convergence analysis of pseudo-transient continuation. *SIAM Journal on Numerical Analysis*, 35(2):508–523, 1998.
- [16] P. K. Khosla and S. G. Rubin. A diagonally dominant second-order accurate implicit scheme. *Computers & Fluids*, 2(2):207–209, 1974.
- [17] D. A. Knoll and D. E. Keyes. Jacobian-free Newton–Krylov methods: a survey of approaches and applications. *Journal of Computational Physics*, 193(2):357–397, 2004.
- [18] L. Mangani, M. Buchmayr, and M. Darwish. Development of a novel fully coupled solver in OpenFOAM: steady-state incompressible turbulent flows. *Numerical Heat Transfer, Part B: Fundamentals*, 66(1):1–20, 2014.
- [19] L. Mangani, M. Buchmayr, and M. Darwish. Development of a novel fully coupled solver in OpenFOAM: steady-state incompressible turbulent flows in rotational reference frames. *Numerical Heat Transfer, Part B: Fundamentals*, 66(6):526–543, 2014.
- [20] L. Mangani, M. Buchmayr, M. Darwish, and F. Moukalled. A fully coupled OpenFOAM solver for transient incompressible turbulent flows in ALE formulation. *Numerical Heat Transfer, Part B: Fundamentals*, 71(4):313–326, 2017.
- [21] F. R. Menter. Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal*, 32(8):1598–1605, 1994.
- [22] F. R. Menter and A. Matyushenko. Generalized k - ω (GEKO) two-equation turbulence model. *AIAA Journal*, 63(11):4590–4606, 2025. doi: 10.2514/1.J065393.
- [23] W. A. Mulder and B. Van Leer. Experiments with implicit upwind methods for the Euler equations. *Journal of Computational Physics*, 59(2):232–246, 1985.
- [24] Y. Notay. An aggregation-based algebraic multigrid method. *Electronic Transactions on Numerical Analysis*, 37:123–146, 2010.
- [25] Y. Notay and P. S. Vassilevski. Recursive Krylov-based multigrid cycles. *Numerical Linear Algebra with Applications*, 15(5):473–487, 2008.
- [26] S. Oliani, N. Casari, and M. Carnevale. ICSFoam: An OpenFOAM library for implicit coupled simulations of high-speed flows. *Computer Physics Communications*, 286:108673, 2023.
- [27] S. V. Patankar and D. B. Spalding. A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows. *International Journal of Heat and Mass Transfer*, 15(10):1787–1806, 1972.

- [28] S. Pollock and L. G. Rebholz. Anderson acceleration for contractive and noncontractive operators. *IMA Journal of Numerical Analysis*, 41(4):2841–2872, 2021.
- [29] S. Pollock, L. G. Rebholz, and M. Xiao. Anderson-accelerated convergence of Picard iterations for incompressible Navier–Stokes equations. *SIAM Journal on Numerical Analysis*, 57(2):615–637, 2019.
- [30] C. M. Rhie and W. L. Chow. Numerical study of the turbulent flow past an airfoil with trailing edge separation. *AIAA Journal*, 21(11):1525–1532, 1983.
- [31] Y. Saad. A flexible inner-outer preconditioned GMRES algorithm. *SIAM Journal on Scientific Computing*, 14(2):461–469, 1993.
- [32] Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2nd edition, 2003.
- [33] T. Uroić and H. Jasak. Block-selective algebraic multigrid for implicitly coupled pressure-velocity system. *Computers & Fluids*, 167:100–110, 2018.
- [34] T. Uroić and H. Jasak. Parallelisation of selective algebraic multigrid for block-pressure-velocity system in OpenFOAM. *Computer Physics Communications*, 258:107529, 2021.
- [35] J. P. Van Doormaal and G. D. Raithby. Enhancements of the SIMPLE method for predicting incompressible fluid flows. *Numerical Heat Transfer*, 7(2):147–163, 1984.
- [36] S. P. Vanka. Block-implicit multigrid solution of Navier–Stokes equations in primitive variables. *Journal of Computational Physics*, 65(1):138–158, 1986.
- [37] H. F. Walker and P. Ni. Anderson acceleration for fixed-point iterations. *SIAM Journal on Numerical Analysis*, 49(4):1715–1735, 2011.
- [38] J. M. Weiss, J. P. Maruszewski, and W. A. Smith. Implicit solution of preconditioned Navier–Stokes equations using algebraic multigrid. *AIAA Journal*, 37(1):29–36, 1999.